Project Report – StudyHelper

Institution:
VIA University College

Course & Semester:
SEP4, 4th Semester

Authors (Student Numbers):
Alexandru Savin(354790)
Cristina Aurelia Matei (354776)
Damian Michal Choina (354789)
Eduard Fekete (355323)
Jakub Maciej Baczek (354814)
Karina Rubahova (354565)
Mara-Ioana Statie (354536)
Marta Zrno (355351)
Piotr Junosz (355502)
Tymoteusz Krzysztof Zydkiewicz (355413)

Supervisors:
Erland Ketil Larsen
Joseph Chukwudi Okika
Kasper Knop Rasmussen
Markéta Tranberg

Date:
May 25, 2026

Character Count:
138,128 characters

TABLE OF CONTENTS

ABSTRACT

Authors: [Name, Name, Name]

[Concise summary of the entire project. State the problem being solved, the three-part technical approach (embedded IoT on ATmega2560, machine learning pipeline, React web frontend), the cloud infrastructure, and the main results. Should stand alone as a paragraph that lets a reader decide whether to read further.]

[TODO: each team check and adjust sentences about their parts of the system so its 100% correct]

StudyHelper is a distributed study-environment monitoring system developed to address the challenge of suboptimal indoor conditions affecting student concentration and academic performance. The system integrates three tightly coupled components: an embedded IoT device based on the ATmega2560 microcontroller that measures temperature, humidity, CO_2 concentration, and light; a machine learning pipeline implemented in Python that predicts a Study Suitability Rating from aggregated sensor data and a React web frontend that presents live readings, historical sensor readings, and ML-generated predictions to the user. All components are deployed as Docker containers on a cloud host managed through Coolify, with the Core API (FastAPI) acting as the central gateway - persisting sensor data in a shared PostgreSQL database and forwarding requests to the MAL FastAPI service for suitability predictions. The IoT firmware communicates with the backend over HTTP using a session-based protocol, transmitting sensor payloads every 30 seconds and keepalive pulses every five seconds. The machine learning models - Random Forest Classifier and Neural Network trained on environmental datasets with MICE imputation - produces both instant and trend based sustainability ratings on a scale of one to five, which are shown to users through the web interface. [Summarise the main quantitative results here, e.g. model accuracy, test coverage, and system uptime, once final figures are available.]

1. INTRODUCTION

Authors: [Name, Name, Name]

1.1 Background and Motivation

[Describe the domain context. What real-world problem does the system address? Why is an IoT-based solution with machine learning relevant here? Who are the stakeholders and what do they need?]

The physical environment in which learning takes place has a measurable impact on cognitive performance. Research has shown that environmental conditions such as temperature, humidity, CO_2 concentration, and lighting directly influence students' ability to concentrate and retain information (Bustamante-Mora et al., 2025) [TODO: add reference in references]. Despite growing awareness of this relationship, most study spaces - libraries, classrooms, dormitories - provide no real-time feedback on whether the ambient conditions are conducive to productive work. Students are left to rely on subjective perception, often noticing that conditions are poor only after their focus has already deteriorated. This makes the problem difficult to handle manually, because students may feel tired or distracted without knowing whether the cause is related to the environment. Objective measurements can help make these conditions visible before they noticeably affect the study session.

The problem is particularly acute in shared environments where individual control over heating, ventilation, or lighting is limited or absent. A student in a crowded library has no practical way of knowing whether the rising CO_2 level in the room is contributing to their fatigue, or whether the ambient temperature is outside the range associated with optimal cognitive function. Existing commercial solutions either focus on a single sensor parameter or require expensive infrastructure that is not feasible in a typical university context. For schools and universities, this creates a practical need for a solution that can monitor indoor conditions without requiring expensive building-wide infrastructure.

By combining low-cost IoT sensing with a machine learning prediction layer, the StudyHelper system closes the gap between raw environmental data and actionable guidance. Rather than simply logging sensor values, the system learns from historical patterns of user-rated study sessions and uses that knowledge to predict how suitable current conditions are likely to be - providing students and teachers with a tool to make informed decisions about when and where to study.

The primary stakeholders are students, who benefit directly from improved self-awareness of their study environment; teachers, who can use condition overviews to assess classroom suitability. [Expand here if additional stakeholder interviews or literature sources are added.]

1.2 Problem Statement

[State the specific problem the project addresses. Be precise about what is currently missing or suboptimal, and what a successful solution would look like.]

Based on the environmental challenges identified in the problem domain, this project addresses the following core question:

Main problem: How can indoor environmental data be monitored and analyzed to better understand how environmental conditions affect the quality of students' study sessions?

This problem is decomposed into the following sub-questions:

- What are the most significant indoor environmental factors that affect students' study conditions?
- What factors can be used to represent study session quality and indoor environmental quality?
- How do students react to changes in indoor environmental quality?
- Is there a measurable relationship between environmental factors and user-provided study ratings?
- How accurately can study suitability be predicted based on collected sensor data?

A successful solution would continuously collect real-world sensor data from study environments, expose this data through a structured API, and deliver ML-driven suitability predictions to students via a responsive web interface. The system should require minimal manual input from the user, apart from connecting a device and optionally submitting a post-session quality rating.

1.3 Project Objectives

[Enumerate concrete, verifiable goals. These will be evaluated against in Results and Discussion. Tie them to the three system components where relevant.

The concrete objectives of the StudyHelper project are as follows:

- **IoT:** The embedded device shall measure temperature, humidity, CO_2 concentration, and light level using the ATmega2560 MCU and transmit structured JSON payloads to the cloud backend via HTTP at a regular interval not exceeding sixty seconds.
- **Cloud Backend:** The Core API (FastAPI) shall act as the central gateway - persisting sensor readings and session metadata in a shared PostgreSQL database and exposing a RESTful API consumed by both the frontend and the MAL service.
- **Machine Learning:** The ML component shall train a classifier capable of predicting a Study Suitability Rating on an integer scale of one to five, using aggregated temperature window features derived from historical sensor data, and expose predictions through a FastAPI endpoint.
- **Frontend:** The React web application shall display live sensor readings and the current ML-predicted suitability rating in a responsive layout that adapts to screen widths of 576 px, 768 px, and 1200 px.
- **DevOps:** All components shall be containerised with Docker, deployed to a public cloud host, and supported by automated CI/CD pipelines on GitHub Actions that enforce unit-test passing and successful build compilation before merging to the main branch.
- **Security:** Communication between the IoT device and the backend shall be protected by [encryption scheme to be specified]; frontend-facing API endpoints shall be protected by [JWT / API key to be confirmed].

[TODO: Review these objectives against the final delivered system before submission and adjust the wording where the implementation diverged from the original plan.]

1.4 Scope and Delimitations

[Define the boundaries. What is explicitly in scope for each component? What has been consciously left out, and why? Important given the 60-page limit.]

The system is designed around a physical IoT device that can be connected to a user account and used in different study environments, such as classrooms, libraries, or personal study spaces. The following delimitations were agreed at project inception and are reflected in the implemented system:

Geographical scope: The device is designed and tested within a campus-related study context, such as classrooms, libraries, or personal study spaces. The IoT device is treated as a portable physical sensor that can be connected to a user account and used in the environment where the user is currently studying. Multi-building deployment is out of scope for this iteration.

Sensor coverage: The system measures temperature, humidity, CO_2 concentration, and light. Sound level measurement is not currently implemented by the active firmware, despite being noted as a candidate sensor in the project description due to the malfunctioning and inaccurate quality of the sound sensor.

Automation: The system provides predictive guidance but does not automate any physical actuators beyond the onboard buzzer alert triggered when the predicted study quality rating reaches the lowest level. Full HVAC or lighting automation is explicitly out of scope.

User model: The system supports user roles such as student who can start and stop sessions via the physical button on the device and submit a post-session quality rating through the frontend, and a teacher who assesses whether conditions are suitable for teaching. Administrator views are out of scope.

Privacy and data collection: The system stores environmental sensor readings and session lifecycle metadata. In addition, user accounts are stored, including name, last name, email address, and a hashed password, as well as optional profile fields such as university, study programme, study year, study goal, and a profile picture reference. Connected device information and post-session ratings submitted by users are persisted so ratings can be associated with the correct device and study session. No audio recordings or video are collected. All personally identifiable data is linked to a user account and is subject to standard data protection considerations; password storage uses hashing.

Medical and psychological analysis: The system does not attempt to measure or infer physiological state, stress levels, or cognitive load directly. The Study Suitability Rating is an environmental proxy, not a clinical assessment.

[Add any additional delimitations agreed with supervisors, particularly regarding scale of the ML dataset or the number of test participants.]

1.5 Related Work

[Survey existing IoT, ML, and web solutions relevant to your domain. Cite with APA. How does your system differ from or build on existing approaches?]

A growing body of research has examined the relationship between indoor environmental quality (IEQ) and cognitive performance. Bustamante-Mora et al. (2025)[TODO: add reference] conducted a case study in university environments demonstrating that temperature and CO_2 levels correlate with measurable changes in student concentration and learning outcomes. Their findings motivate the

sensor selection in this project, specifically the inclusion of CO_2 as a key environmental indicator alongside temperature and humidity.

IoT-based classroom monitoring systems have been explored in several contexts. [Cite one or two relevant prior IoT classroom studies here, e.g. systems based on Raspberry Pi or Arduino platforms.] These systems typically focus on data logging and threshold-based alerting rather than predictive modelling, which is the distinguishing contribution of the StudyHelper approach.

In the domain of machine learning for indoor environment prediction, [cite relevant ML papers on predicting thermal comfort or cognitive performance from environmental data]. The use of ensemble methods such as Random Forest[TODO: update after final model choice] for environmental classification has been validated in comparable contexts and supports the model choice made in this project.

On the frontend side, React-based dashboards for IoT data visualisation are a well-established pattern. [Reference any relevant existing open-source dashboards or comparable student-facing tools.]

StudyHelper differs from prior work in its end-to-end integration: rather than treating sensing, prediction, and presentation as separate research artefacts, the project combines all three within a single deployable system. The use of a user-provided post-session rating as the supervised learning target - rather than a physiologically measured focus indicator - is a pragmatic design choice that makes the system trainable without invasive data collection, and represents a novel framing of the suitability prediction problem.

[This section should be expanded with full APA citations before submission. The references listed here are indicative placeholders.]

2. ANALYSIS

Authors: [Name, Name, Name]

2.1 Domain Model

[Describe the key concepts and relationships in the problem domain. Reference the domain model diagram:]

[Walk through the most important entities and associations. Justify the key modelling decisions.]

The domain model captures the core concepts of the StudyHelper system and the relationships between them. The central entity is the **StudyEnvironment**, which represents the physical room or space being monitored. A StudyEnvironment has measurable attributes - temperature, humidity, CO_2 level, light level, and noise level - as well as derived attributes: a current suitability level and a predicted suitability level .

Two actors interact with the StudyEnvironment: the **Student**, who monitors conditions to decide whether to begin or continue a study session, and the **Teacher**, who assesses whether conditions are suitable for teaching. Both actors share a common monitoring relationship with the StudyEnvironment; however, only the Student can submit a session rating, which feeds back into the ML training pipeline.

The **EnvironmentConditionHistory** entity records time-stamped snapshots of sensor readings. Each snapshot is associated with a session, allowing the system to aggregate readings over the duration of a study session and compute window-level statistics (minimum, maximum, mean) used as ML features. This association is represented as a one-to-many relationship from StudyEnvironment to EnvironmentConditionHistory.

At the persistence layer, the domain model maps to three database tables: `devices`, `sessions`, and `data`. The `devices` table identifies each physical IoT unit by a string ID. The `sessions` table records the lifecycle of a study session, including start time, end time, last-pulse timestamp, and the optional post-session study quality integer. The `data` table stores individual sensor readings linked to a session.

The key modelling decision is the separation of the session concept from the raw data concept. By attaching individual sensor readings to a session rather than to a device directly, the system can compute per-session aggregates for ML feature extraction without requiring complex time-windowing logic in the database. [Describe any domain model revisions made after initial analysis here.]

2.2 User Stories

Two system sequence diagrams capture the primary interaction flows in the StudyHelper system. [TODO: create 2 SSD for example those and change the description]

SSD 1 - Study Session Lifecycle (IoT-initiated)

When the student presses Button 1 on the device, the firmware sends `POST /device` to register the device identifier, then `POST /session` to create a new active session. The backend persists both records and returns a session ID, which the firmware stores in memory. During the session, the firmware fires two software timers: a five-second pulse timer and a thirty-second data timer. On each pulse, the firmware sends `PATCH /session/{id}/pulse` so the backend can detect abandoned sessions. On each data tick, the firmware reads all sensors and sends `POST /data` with a JSON payload containing temperature, humidity, light level, and CO_2. The backend stores the reading and may return a `study_quality` field in the response; if the value is 1 (worst), the firmware triggers the buzzer. When Button 1 is pressed again, the session is closed locally by clearing the session ID, and no further pulses or data are sent.

SSD 2 - Frontend Condition Overview and Prediction (frontend-initiated)

The student opens the web application and the React frontend issues `GET /data` to the IoT backend to retrieve the most recent sensor readings. It then issues `POST /predict` to the MAL FastAPI service, passing the current and windowed temperature values. The MAL service loads the trained Random Forest model and returns a predicted Study Suitability Rating as a JSON integer. The frontend renders the readings and the rating on the dashboard. This cycle repeats at a configurable polling interval, defaulting to once every 30 seconds to match the IoT transmission cadence.

[Reference the SSD diagram files in the `Documentation/Analysis/ssds/` directory once they are exported to SVG. If additional SSDs were created for post-session rating submission or for the Teacher condition overview, add them here.]

3. DESIGN

3.1 Design Overview

Authors: [Name, Name]

3.1.1 System Architecture

[Describe the overall architecture: IoT device → Cloud backend → Frontend, with the ML pipeline integrated into or alongside the cloud layer. Reference the architecture diagram:]

[Justify the key architectural decisions: why this decomposition, how components communicate (protocols, data formats), and how data flows end-to-end.]

The StudyHelper system follows a layered, distributed architecture comprising four runtime components: the embedded IoT firmware, the Core API, the MAL API, and the React frontend. All server-side components are deployed as Docker containers behind a Coolify-managed reverse proxy (Caddy/Traefik) and share a single PostgreSQL database, as described by `docker-compose.yml`.

The IoT device sits at the edge of the system. It collects sensor readings and transmits them over HTTP to the Core API, acting as the sole source of real-world environmental data. This unidirectional push pattern was chosen because the device does not need to receive commands in normal operation and because HTTP over TCP is straightforward to implement with the ESP8266-based Wi-Fi module used alongside the ATmega2560.

The Core API (FastAPI) is the system's central gateway. It handles device registration, session lifecycle management, sensor data persistence, ratings and user authentication. It is the only component with direct write access to the PostgreSQL database. When new sensor data is received, the Core API can call the MAL API to obtain a predicted study suitability rating and store the result together with the sensor reading. This keeps the frontend connected to one API surface while allowing the MAL service to be retrained and redeployed independently without affecting the Core API.

The MAL API (FastAPI) is responsible for model inference and data collection for retraining. It receives prediction requests forwarded by the Core API, loads the trained model artifact, and returns a suitability rating. It also exposes endpoints for exporting raw sensor data from the database to support offline retraining workflows.

The React frontend is a static single-page application served by Nginx inside a Docker container. It communicates exclusively with the Core API via REST calls - user authentication, profile/device data, session state, dashboard readings, sensor history, calendar data, and ratings. The frontend does not calculate predictions itself, it displays the predicted_study_quality value returned by the Core API. All inter-service communication uses JSON over HTTP. The Coolify reverse proxy handles TLS termination and routes public traffic to the frontend and Core API containers.

Data flows through the system as follows: the IoT device pushes a sensor payload to the Core API during an active session; the Core API persists the reading in PostgreSQL and requests a suitability prediction from the MAL API; the MAL API returns a rating; the Core API stores the rating together with the sensor data and relays the result to the frontend. The frontend polls the Core API for dashboard data and renders the latest readings, chart, and recommendation.

3.1.2 Cloud Architecture

The StudyHelper backend is hosted on **Coolify**, an open-source self-hosted PaaS running on a public VPS. Coolify provides container lifecycle management and a built-in reverse proxy (Caddy/Traefik) that terminates TLS and routes public traffic to the frontend and Core API services. Internal traffic between services stays on the Docker network.

Containerisation: All runtime services are defined in `docker-compose.yml` and run as Docker containers: `postgres`, `api`, `mal-api`, and `frontend`. The `api` service is built from `API/Dockerfile` (Python 3.12) and serves the Core API with Uvicorn on port 8080. The `mal-api` service is built from `MAL/Dockerfile` and serves the ML inference API on port 8000, including the committed models artifacts. The `frontend` service is built from `Frontend/Dockerfile`, compiles the Vite/React application, and serves static assets with Nginx on port 80. The `postgres` container uses the official `postgres:16-alpine` image with a named volume (`postgres_data`) for persistence.

Database: A single PostgreSQL 16 instance is the shared persistence layer. The schema is initialized via `initdb/01_schema.sql` mounted as an entrypoint init script. The Core API owns write access for devices, sessions, and sensor data, while the MAL API reads data for export and model support.

RESTful API design: The IoT device communicates directly with the Core API over HTTP (no message queue is used). Device registration uses `POST /device`, sessions use `POST /session` and `PATCH /session/{id}/pulse`, and sensor submissions use `POST /data`. The frontend communicates exclusively with the Core API for readings and predictions; prediction requests are forwarded by the Core API to the MAL API `POST /predict` endpoint. All inter-service payloads are JSON.

Serverless workloads: No separate serverless runtime is deployed. Data extraction and model verification tasks are handled by on-demand GitHub Actions workflows (for example, `data-extract.yml`).

Continuous delivery: On every push to `main`, a GitHub Actions workflow sends a webhook to Coolify, which pulls updated images from GHCR and restarts the affected containers. Pre-built images for `mal-api` and `frontend` are published to GHCR as part of the MLOps and frontend CI/CD pipelines.

3.1.3 Security Design

Authors: [Cristina Matei]

For frontend-facing API endpoints, the Core API uses JWT-based authentication. After a successful login, the backend creates a JWT signed with `SECRET_KEY` and stores it in an `HttpOnly` cookie named `access_token`. Protected endpoints read the token from this cookie through the shared `get_current_user` dependency. Bearer-token support is still kept as a fallback, but the frontend no longer stores the JWT in `localStorage`.

The frontend sends authenticated requests with `credentials: "include"` so the browser includes the authentication cookie automatically. This is used by protected API calls such as dashboard, profile, calendar, session, device, and rating requests. Logout clears the authentication cookie on the backend.

The JWT signing key is read from the `SECRET_KEY` environment variable instead of being hard-coded in the source code. The variable is documented in `.env.example` and passed into the API container through `docker-compose.yml`. This makes the signing key configurable per environment and avoids committing the real secret to Git.

Additional considerations: Login rate limiting is implemented to reduce repeated brute-force login attempts. The API allows up to five failed login attempts within a 15-minute window for the same client/email combination. After the limit is reached, login returns HTTP 429 until the window expires. CORS is configured to allow the frontend origins used in local development and to support credentials, which is required because authentication uses cookies.

Calendar events are also protected by the logged-in user identity. The calendar endpoints use the current authenticated user from the JWT cookie, and every create, read, update, and delete operation is linked to `current_user["id"]`. This means users only see and modify their own calendar events, not events created by other users.

3.1.4 DevOps Strategy

3.2 IoT Design

Authors: [Jakub Maciej Baczek, Tymoteusz Krzysztof Zydkiewicz]

The IoT component is the physical sensing layer of the StudyHelper system. Its main responsibility is to collect indoor environmental data from the study space and transmit it to the backend, where the readings can be stored, displayed in the frontend, and used as input for machine-learning-based study suitability prediction.

The design is based on three central requirements. First, the device must measure the environmental factors selected for the project: temperature, humidity, CO2 concentration, and light level. Second, it must support a simple study-session workflow controlled through physical buttons, so the user can start and stop monitoring without interacting with the web interface. Third, it must communicate with the backend using structured HTTP requests over Wi-Fi.

The IoT design therefore combines sensors, user input, local status feedback, and network communication in one embedded prototype. The firmware is divided into small modules with clear responsibilities, so that hardware drivers, backend communication, display status logic, and the main session controller can be developed and tested separately.

3.2.1 Hardware Architecture

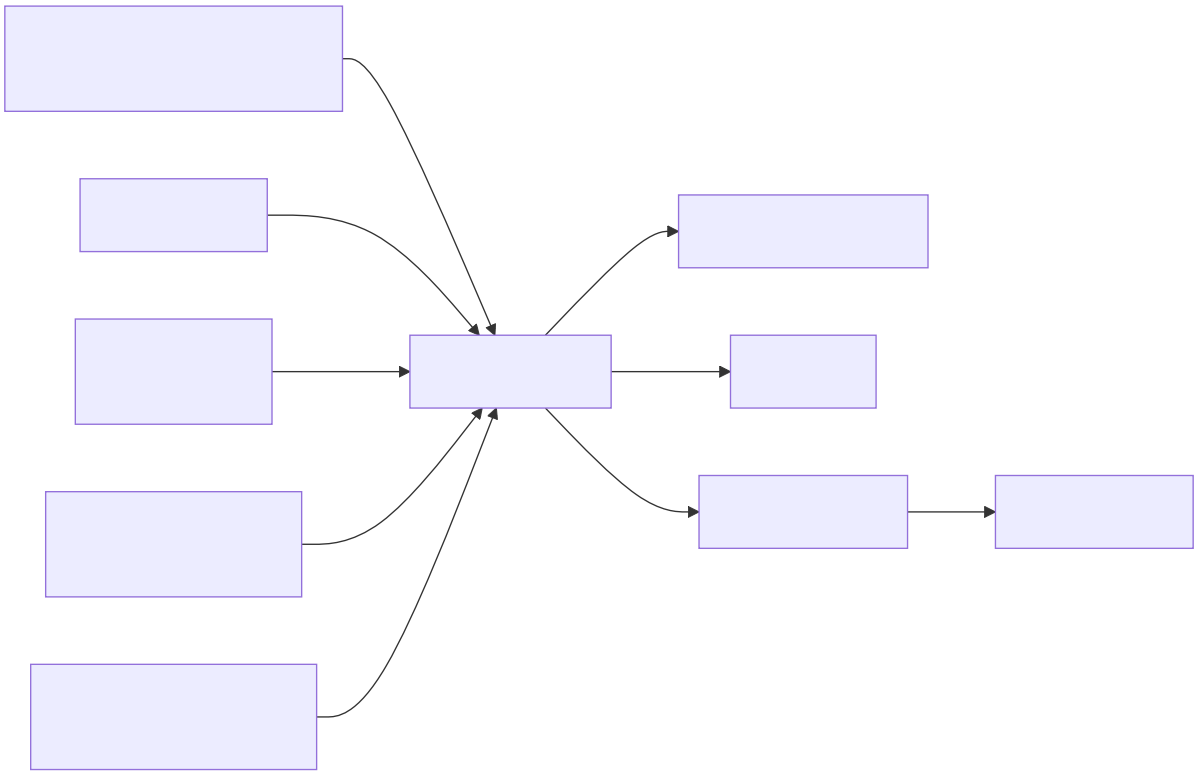


Figure 3.x: Hardware block diagram of the IoT device.

The hardware architecture is centred around an ATmega2560-based microcontroller board. The ATmega2560 was chosen because it provides sufficient GPIO pins, ADC channels, timers, and UART interfaces for a multi-sensor embedded prototype, while remaining compatible with the Arduino and PlatformIO development environment used in the project.

The device uses the following main hardware components:

Component	Interface	Purpose
ATmega2560 microcontroller	GPIO, ADC, UART, timers	Main controller for sensors, buttons, display, buzzer, and communication
DHT11 temperature and humidity sensor	Single-wire digital GPIO	Measures air temperature and relative humidity

Component	Interface	Purpose
MH-Z19B CO2 sensor	UART3 at 9600 baud	Measures CO2 concentration in ppm
KY-018 light sensor	ADC channel PK7	Measures ambient light level
Wi-Fi module	UART / AT commands	Provides TCP/IP communication with the backend API
Button 1	GPIO input with pull-up	Starts and stops a study session
Button 2	GPIO input with pull-up	Triggers an instant measurement mode
Four-digit display	Shift-register style GPIO output, refreshed by timer interrupt	Shows local device state
Buzzer	GPIO output	Alerts the user when predicted study quality is poor

The selected sensors match the environmental factors used by the StudyHelper system. The DHT11 provides temperature and humidity, the MH-Z19B provides CO2 concentration, and the KY-018 light sensor provides a simple analogue measure of room brightness. The light driver inverts the raw ADC value so that higher values represent brighter conditions, making the reading easier to interpret in the rest of the system.

The CO2 sensor requires a different interaction pattern from the DHT11 and light sensor. The firmware sends a read command to the sensor and receives a nine-byte UART response frame. The frame is validated with a checksum before the measured ppm value is accepted. This design avoids using invalid or corrupted CO2 readings in the transmitted sensor payload.

The device also includes local user interaction and feedback. Button 1 controls the study-session lifecycle. Button 2 is used for instant measurement mode when no session is active. The four-digit display shows status patterns for boot, idle, active-session, and instant-measurement states. The buzzer is used as an alert mechanism when the backend returns the lowest study quality rating.

Sound measurement was considered during the project, but it is not part of the final active IoT design because the available sound sensor did not provide reliable enough readings. The final hardware design therefore focuses on temperature, humidity, CO2, and light, which are the sensor values actually transmitted by the firmware.

3.2.2 Embedded Software Architecture

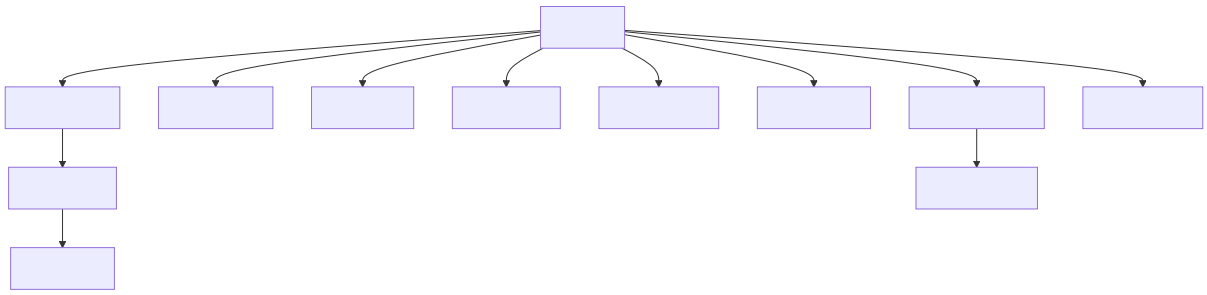


Figure 3.x: Firmware module structure for the IoT component.

The embedded software is implemented in C for the ATmega2560 and follows a modular architecture. The `main.c` file coordinates the application flow, while individual modules handle backend communication, HTTP request construction, display status patterns, sensors, buttons, timers, and local alerts.

The most important firmware modules are:

Module	Responsibility
`main.c`	Coordinates boot, Wi-Fi setup, button handling, timers, session state, and sensor reading
`server_api.c` / `server_api.h`	Implements backend-specific actions such as device registration, session creation, keepalive pulses, data upload, instant prediction requests, and local alert handling
`wifi_http.c` / `wifi_http.h`	Resolves the backend host, creates TCP connections, builds HTTP requests, transmits payloads, and reads responses
`dht11.c`	Reads temperature and humidity from the DHT11 sensor
`co2.c`	Handles UART communication and checksum validation for the CO2 sensor
`light.c`	Wraps ADC access for the light sensor
`button.c`	Reads physical button states with pull-up inputs
`timer.c`	Provides software timers used for periodic session actions

Module	Responsibility
`display_status.c`	Maps firmware states to four-digit display patterns
`buzzer.c`	Produces local audio alerts

The firmware uses a cooperative superloop design. After startup, the program continuously checks button states and timer flags. Time-critical and periodic behaviour is represented by flags such as `pulse_due` and `data_due`, which are set by timer callbacks and then handled in the main loop. This keeps the main control flow explicit and avoids introducing a full real-time operating system, which would be unnecessary for the scope of the device.

Three software timers define the main runtime schedule:

Timer	Interval	Purpose
Pulse timer	5 seconds	Sends a keepalive pulse while a session is active
Data timer	30 seconds	Sends environmental measurements while a session is active
Button cooldown timer	10 seconds	Prevents accidental repeated start/stop actions

The device communicates with the backend using HTTP over TCP through the Wi-Fi module. HTTP was chosen because the rest of the StudyHelper system exposes REST-style endpoints and JSON payloads. This keeps the interface between the IoT firmware and backend simple, transparent, and easy to test. Each request opens a TCP connection, sends an HTTP request, waits for a response for up to three seconds, and then closes the connection.

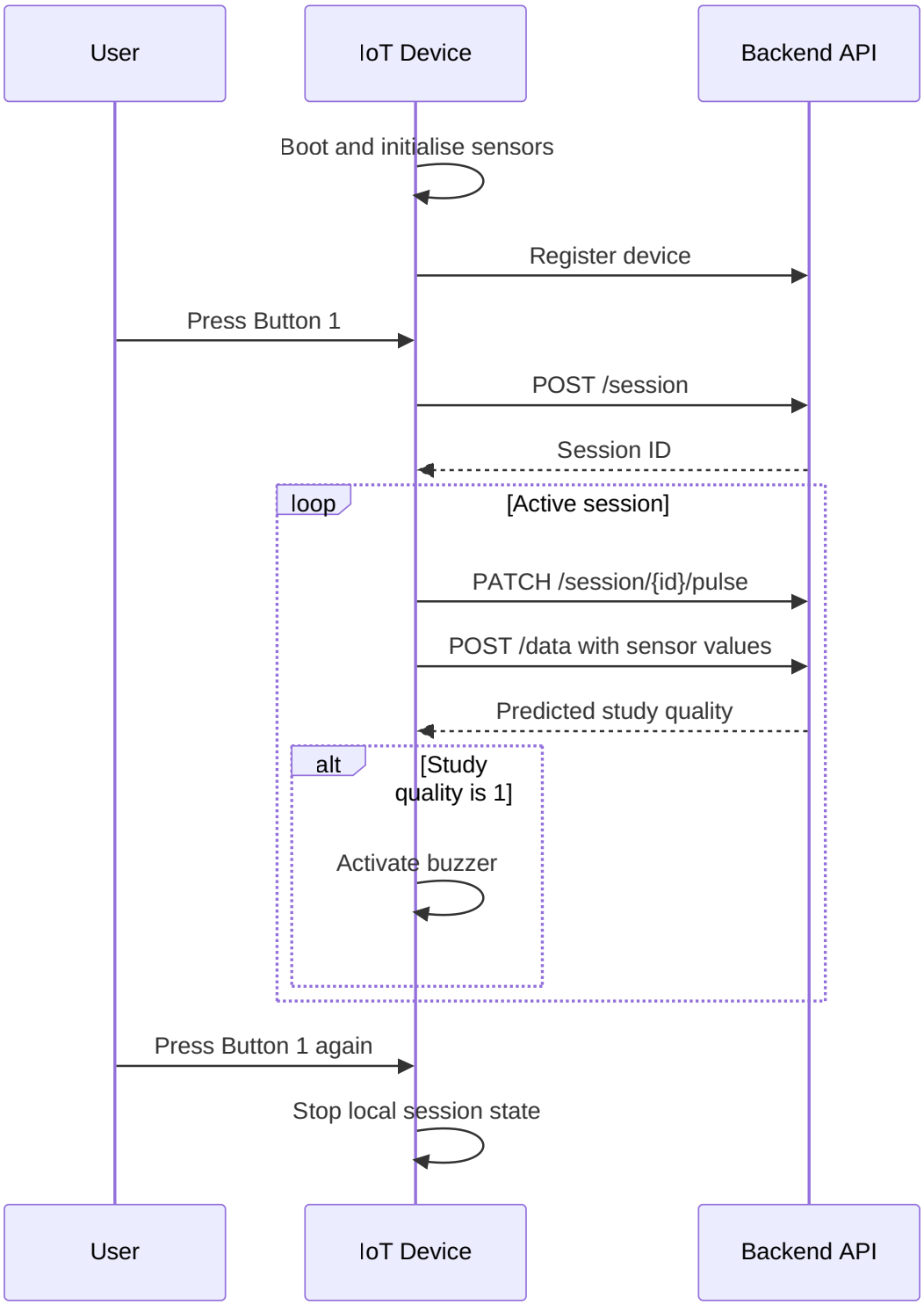


Figure 3.x: Session flow between the user, IoT device, and backend API.

The session flow begins during boot. The firmware initialises the display, UART, sensors, Wi-Fi module, and backend host resolution. It then registers the physical device ID with the backend. After this setup, the device enters idle mode and waits for user input.

When Button 1 is pressed, the firmware sends a `POST /session` request containing the configured device ID. If the backend returns a valid session ID, this ID is stored locally and used for later requests. During an active session, the device sends `PATCH /session/{id}/pulse` every five seconds and `POST /data` every thirty seconds. The data payload contains the session ID, temperature, humidity, light level, and CO2 level in JSON format.

The firmware stores request and response data in statically allocated buffers. This design avoids dynamic memory allocation and reduces the risk of heap fragmentation on the ATmega2560. Watchdog resets are performed during network waiting periods so that normal communication delays do not incorrectly reset the device, while still protecting the firmware from longer hangs.

If the backend reports that a session is no longer alive, the firmware clears the local session ID and attempts to start a new session. If a sensor data response contains a study quality rating of `1`, the buzzer is activated to warn the user that the current conditions are poor. This creates a local feedback loop where the IoT device does not only collect data, but also reacts to the study suitability prediction returned by the system.

The firmware also includes an instant measurement mode controlled by Button 2. This mode allows the device to take a one-time measurement and request a prediction without entering the normal periodic session loop. It was designed as a quick check of the current environment and reuses the same sensor-reading and backend-communication modules as the session workflow.

3.3 Machine Learning — Data Exploration

Authors: [Name, Name]

3.3.1 Data Sources and Collection Strategy

The initial search for data focused on the relationship between environmental noise and cognitive focus. We explored combining datasets describing focus-related effects of background noise with labeled sound categories from WAV files, extracting frequencies and loudness. However, as the project evolved, we narrowed the ML objective from direct focus prediction to predicting a user-provided **Study Suitability Rating**. This shift was necessary because “focus” is a subjective internal state that cannot be

directly measured by our sensors. By using a user-provided rating, we anchored our target variable in observable environmental conditions and explicit user feedback. [...]

3.3.2 Exploratory Data Analysis

During the exploratory phase, several candidate datasets were evaluated. We identified and eliminated datasets that appeared synthetic or "too perfect" to be realistic sensor data. For example, in datasets labeled as "Data 2" and "Data 4," the distributions of humidity, noise, and light were suspiciously uniform or perfectly bell-shaped, lacking the stochastic noise typical of real-world environments.

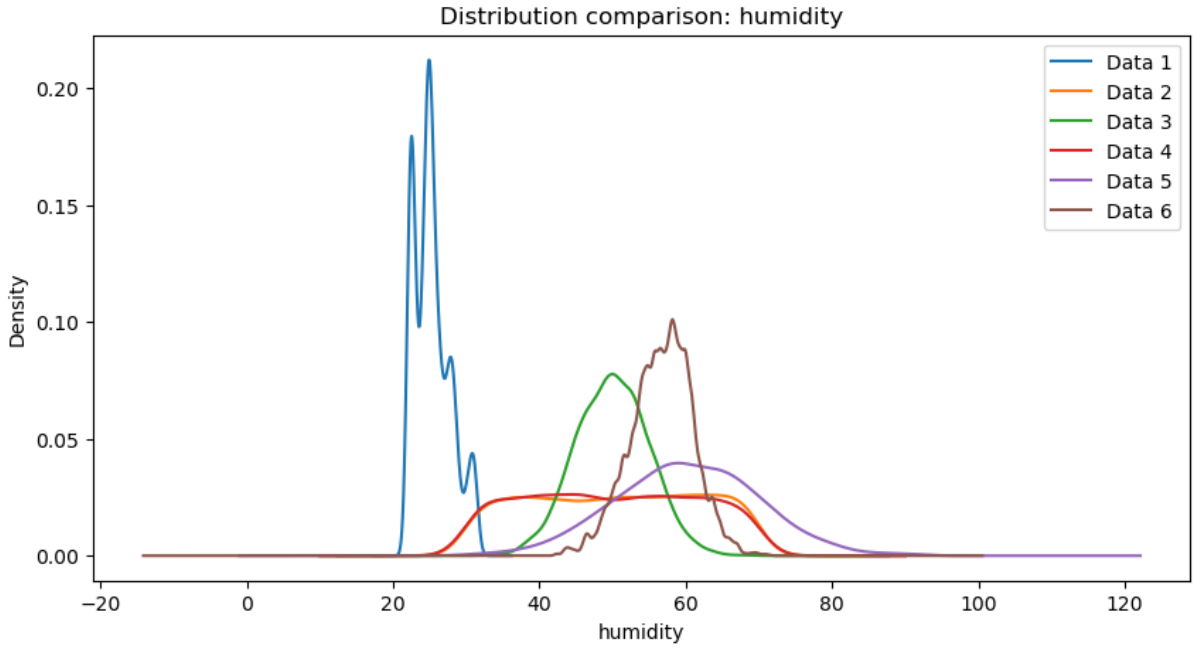


Figure 1: Feature Distributions of Suspicious Datasets

Figure 3.x: Suspiciously perfect feature distributions in candidate datasets.

Correlation analysis further revealed insights into data quality. Healthy datasets exhibited natural correlations between temperature, CO2, and humidity. Conversely, some "suspicious" datasets showed near-zero correlation across all features, suggesting high randomness or artificial generation.

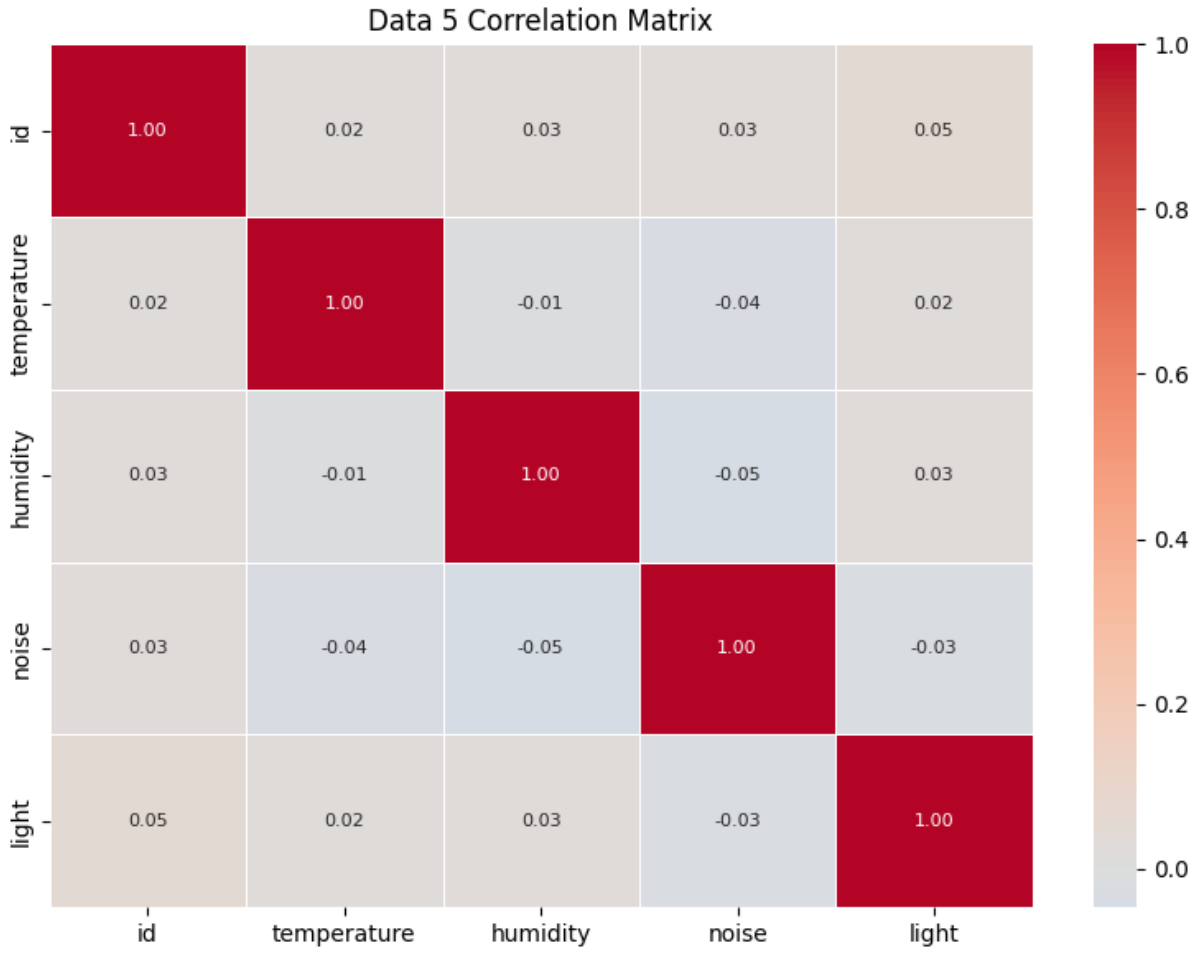


Figure 2: Healthy vs Suspicious Correlation Matrices

Figure 3.y: Correlation matrices showing healthy physical relationships (left) vs suspicious randomness (right).

3.3.3 ML Problem Formulation

The ML task is formulated as a supervised learning problem aimed at predicting the Study Suitability Rating. Specifically, this is defined as a **Multi-Class Classification** problem:

- **Inputs (X):** The features are derived from continuous, time-series sensor telemetry (Temperature, Humidity, CO2, and Light). These raw readings are dynamically aggregated into structured vectors (capturing current, minimum, maximum, and mean values) to summarize the state and variance of the environment over an active session.

- **Output (Y):** The target variable is the Study Suitability Rating, consisting of discrete integer classes ranging from 1 (poor suitability) to 5 (excellent suitability).
- **Objective:** The goal is to learn the complex, non-linear relationships between the physical characteristics of a room and subjective human sustainability raiting. By mapping environmental metrics to historical user ratings, the model enables the system to automatically predict the quality of an environment in real-time.

3.4 Frontend Design

Authors: [Cristina Matei]

3.4.1 UI/UX Design

The frontend was built for students who want to quickly check if their study environment is suitable. The most important actions are logging in, connecting a device, seeing the current sensor values, checking previous measurements, and rating a study session afterwards. Because of this, the dashboard is the main page after login, while the profile and calendar pages are available from the navigation bar.

The application is split into a few main views. The login and register pages are used before the user enters the system. The dashboard shows live sensor readings, historical data, a short recommendation and the current predicted study quality as both a numeric suitability value in the sensor cards and as a prediction line in the chart. The profile page is used for user information, password changes, and connecting the physical device. The calendar page gives the user a simple place to plan study events. This keeps the monitoring part easy to access, while settings and profile information are kept separate.

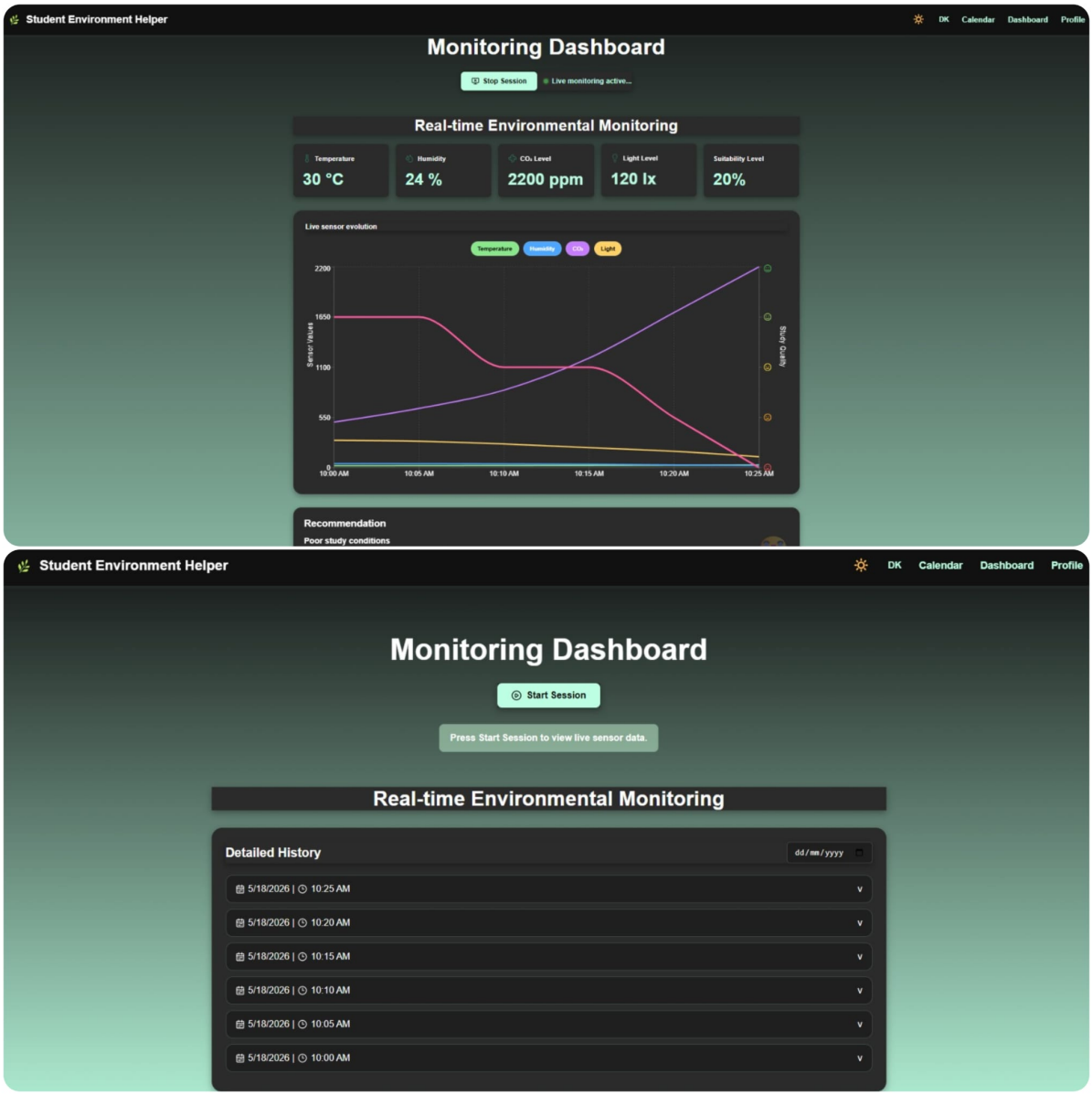


Figure 3X: Figure 3X: Dashboard states during an active study session and before. The active session view shows live readings, the chart, recommendations, and session controls after the frontend attaches to an active IoT session; the second view keeps live sensor cards and the chart hidden while history remains available.

3.4.2 Frontend Architecture

The frontend is implemented with React and Vite. React is used to build the interface from reusable components, and Vite is used for local development and for creating the production build. The

application starts in `src/main.jsx`, where the React app is rendered and wrapped in `BrowserRouter`, `ThemeProvider`, and `LanguageProvider`. The routes are defined in `src/App.jsx`, where public routes such as login/register and protected routes such as dashboard, profile, and calendar are configured.

The frontend uses React Context API together with local component state managed through `useState`. Context providers are used for global state such as language selection and theme management, while page-specific state such as form input, loading states, selected ratings, and dashboard session state is kept inside the relevant components. Communication with the backend API is handled through service modules using the Fetch API. These service modules keep API calls separate from the page components and make the component code easier to read.

The source code is divided into folders by responsibility:

- `components/` contains reusable UI parts and route helpers. This includes `Navbar`, `ProtectedRoute`, `PublicRoute`, `SensorCard`, `SensorChart`, `SessionRating`, `LoadingSpinner`, and `EmptyState`.
- `pages/` contains the main screens of the application, such as `LoginPage`, `RegisterPage`, `Dashboard`, `Profile`, and `CalendarPage`. Some page-specific tests are also placed here.
- `services/` contains the functions that communicate with the backend. For example, authentication, dashboard data, profile updates, device connection, session lookup, and rating submission are handled here.
- `context/` contains state that is needed in several places. `LanguageContext` handles the selected language, and `ThemeContext` handles light and dark mode.
- `translations/` contains the text used for English and Danish labels. The components use the `t` object from the language context instead of hardcoding all text directly.
- `tests/` contains the shared test setup and frontend tests. Page-specific tests are located together with their related pages inside the `pages/` folder.

Routing is handled with `react-router-dom`. The public routes are `/login`, and `/register`. These routes use `PublicRoute`, so a user who is already logged in is redirected to `/dashboard`. The protected routes are `/dashboard`, `/profile`, and `/calendar`. These routes use `ProtectedRoute`, which redirects the user to `/login` if no user is stored. Unknown routes (path="*") are redirected to the dashboard. This gives a simple separation between pages that can be opened before login and pages that require authentication.

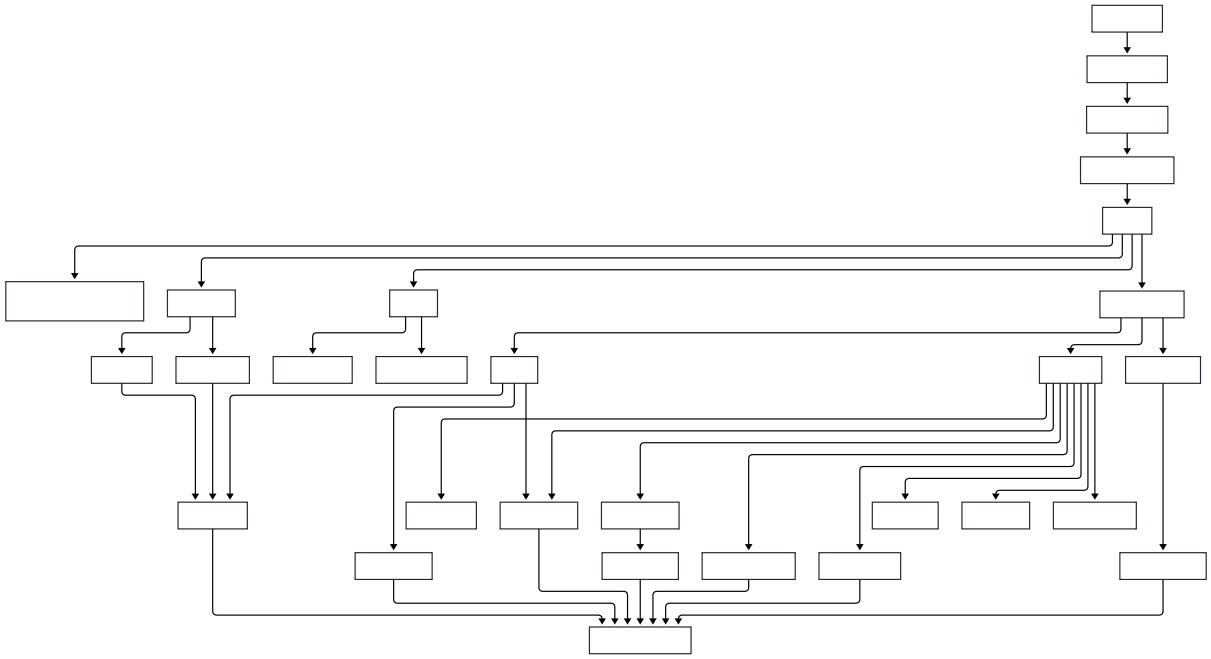


Figure 3X: Frontend component structure showing the main providers, routes, pages, reusable components, and service layer.

3.4.3 Responsiveness Strategy

The frontend uses custom CSS with Flexbox and CSS Grid. No large UI framework was used, because the pages needed fairly specific layouts for the dashboard, profile page, and navigation. Sensor cards and profile fields use grid layouts, while buttons, forms, and navigation areas mostly use flex layouts.

The layout was made to work at the required widths of 576 px, 768 px, and 1200 px. On smaller screens, the dashboard content stacks vertically so that the user can still read the sensor cards, chart, recommendation, and history. On medium screens, the cards and profile fields can use more columns. On larger screens, the dashboard has more space for the chart and the detailed history list. The goal was to keep the same features available on both phones and laptops, without requiring horizontal scrolling.

3.5 IoT Implementation

Authors: [Damian Michal Choina]

3.5.1 Sensor and Actuator Drivers

The firmware uses a layered driver structure: each peripheral is encapsulated behind a small header in `IOT/lib/` that exposes a minimal initialisation function and one or two read or write operations. This separation keeps the application code in `main.c`, `server_api.c`, and `wifi_http.c` free of register-level concerns and makes individual drivers replaceable without touching the application logic.

Four sensors and three actuator-class peripherals are integrated. The **DHT11** temperature and humidity sensor exposes a blocking call `dht11_get()` that returns four values — humidity integer and decimal parts, temperature integer and decimal parts — along with a status code, so the application can skip transmission on a failed read. The **MH-Z19B** CO_2 sensor communicates over UART and is driven by two separate calls: `co2_request_measurement()` triggers a new reading, and `co2_read_ppm()` retrieves the latest verified value once available. Because the sensor needs time to respond between request and reply, the two calls are scheduled on alternating data cycles in the main loop - if a fresh reading is not yet available, the last cached value is reused so the transmission cadence is never blocked by sensor latency. The **KY-018** light sensor is read via `light_measure_raw()`, which returns a 10-bit value already scaled so that low means dark and high means bright. Moreover, no further calibration is applied in firmware, since interpretation is left to the server.

The remaining peripherals are simpler. **Two pushbuttons** are polled via `button_get()` on every iteration of the main loop and debounced via a 50 ms re-read pattern rather than dedicated interrupts. A **four-digit 7-segment display** is driven by the low-level `display_setValues()` and `display_setDecimals()` functions; on top of these, the `display_status` module — written as part of this project — exposes four named patterns (`boot`, `idle`, `session`, `instant`) so the application does not have to know about individual digit codes. A **buzzer** is exposed as a single `buzzer_beep()` call producing a short tone; longer alerts are produced by repeating the call, used to signal unfavourable study conditions — the buzzer sounds when the server returns a `study_quality` value below 4. Finally, a **software-timer abstraction** in `timer.h` provides callback-based timers, allowing the application to register callbacks for pulse, data, and button-cooldown events without managing hardware timers directly.

The CO_2 scheduling pattern in the main loop illustrates how the application accommodates sensor-side latency without blocking. On every data cycle, the most recent reading is consumed (if one has become available), the cached value is updated, and a new measurement is immediately requested for the next cycle:

```
        if (co2_read_ppm(&current_co2) == CO2_OK) {
            latest_co2_ppm = current_co2;   /* fresh reading — update cache */
        } else {
```

```
            /* no new reading yet — reuse last cached value */
        }
        co2_request_measurement();          /* request next reading */
```

3.5.2 Cloud Communication Implementation

Network communication is split into two layers. The lower layer (`wifi_http.c`) is responsible for the HTTP transport — DNS resolution, TCP connection lifecycle, and request formatting. The upper layer (`server_api.c`) is responsible for protocol concerns — endpoint paths, JSON payloads, response parsing, and session state. This separation keeps the HTTP layer reusable and the protocol layer free of low-level transport details.

At boot, `http_resolve_host()` calls `wifi_command_get_ip_from_URL()` once to translate `SERVER_HOST` into an IPv4 address, which is then cached in a static buffer so each subsequent request avoids a DNS round-trip. The main loop retries DNS until it succeeds, treating it as a hard prerequisite for normal operation. Each HTTP request is built into a single 384-byte stack buffer using `snprintf` and handed to the Wi-Fi driver via `wifi_command_TCP_transmit()`. A fixed-format request is used (HTTP/1.0, `Connection: close`, JSON body):

```
        snprintf(req, sizeof(req),
                 "%s %s HTTP/1.0\r\n"
                 "Host: %s:%d\r\n"
                 "Content-Type: application/json\r\n"
                 "Content-Length: %u\r\n"
                 "Connection: close\r\n"
                 "\r\n%s",
                 method, endpoint, SERVER_HOST, SERVER_PORT,
                 (uint16_t)strlen(body), body);
```

Failures are handled defensively at every step. A failed TCP connect or transmit closes the connection, waits 500 ms with watchdog resets, and returns the error to the caller. After a successful send, the loop busy-waits up to 3 000 ms for the server's response — again with watchdog resets — and proceeds even if no response arrives, so the device never deadlocks on a silent server. All buffers are statically allocated; nothing on the network path uses the heap.

At the protocol layer, `server_api.c` implements three workflows: one-time device registration (`POST /device`), session lifecycle (`POST /session`, `PATCH /session/{id}/pulse`), and data reporting (`POST /data`, `POST /instant-measurement`). Session start is wrapped in a retry loop of up to five attempts with a 2-second back-off and if the server cannot be reached or its

response cannot be parsed within that budget, the watchdog is deliberately enabled with a short timeout to force a clean reboot. This recovery pattern — retry, then reboot — was chosen because a partially-initialised device with no session ID has no meaningful path forward. Pulse responses are also inspected: if the server replies with `"alive":false`, the device assumes the session was lost and transparently restarts it without user intervention.

3.5.3 Main Application Logic

The main application is a single cooperative loop in `main.c`. After hardware and Wi-Fi initialisation are complete and the device has registered with the backend, the loop runs forever and performs four responsibilities on every iteration: poll the buttons, dispatch any button action, check time-driven flags, and dispatch the corresponding periodic server transmission.

Two pieces of time-driven behaviour are required during an active session: a pulse every 5 seconds to keep the server-side session alive, and a data submission every 30 seconds carrying current sensor readings. Rather than performing these transmissions directly from interrupt handlers — where blocking HTTP calls would be unsafe — the software timer system raises two volatile flags, `pulse_due` and `data_due`, which are consumed at a safe point in the main loop:

```
        if (session_active && !request_in_progress && data_due) {
            data_due = 0;
            request_in_progress = 1;
            /* read sensors, then server_send_data(...) */
            request_in_progress = 0;
        }
```

The `request_in_progress` flag acts as a simple mutex. While one HTTP request is in flight, no other request can be started and no button press can interrupt it. This avoids re-entering the Wi-Fi driver with overlapping commands, which the underlying ESP8266-based module does not tolerate.

User input is handled in the same loop. Button 1 toggles between the idle and active session states; Button 2 triggers an on-demand instant measurement and is only operative outside an active session. Both buttons are debounced via a two-read pattern with a 50 ms delay between samples. An additional 10-second cooldown timer prevents the user from rapidly toggling the session on and off, both because the underlying HTTP work is expensive and to give the server's session lifecycle time to settle. State transitions are mirrored on the 7-segment display through the `display_status` module — `boot`, `idle`, `session`, and `instant` — so that the visible state of the device is always consistent with its internal state without scattering display logic across the file.

3.6 Machine Learning — Preprocessing and Pipeline

Authors: [Name, Name]

3.6.1 Data Cleaning and Imputation

A significant challenge was merging disparate datasets, which often resulted in missing columns for specific sensors (e.g., noise or light). To handle these missing values while preserving natural variance, we implemented a sophisticated imputation strategy using the **Multivariate Imputation by Chained Equations (MICE)** framework.

Rather than using simple means or linear regression, we adopted a cluster-based approach:

1. **Environment Type Clustering**: We used k-means clustering to group data points into "environment types" based on features that were fully present (Temperature, CO2, Humidity). This accounts for different physical environments (e.g., sun-exposed sessions vs. windowless labs sessions) where sensor correlations might differ.
2. **ExtraTrees Estimator**: Within the MICE framework, we utilized an ExtraTrees estimator to model non-linear relationships.
3. **Variance Preservation**: We modified the imputation logic to include natural distribution variance based on the average standard deviation from the trees, preventing the "flat average" effect.

If a cluster suffered from extreme sparsity (e.g., completely missing a feature like noise), a global median was used as a fallback to prevent model bias.

3.6.2 Feature Selection

Our feature extraction pipeline aggregates ("linearize") time-series sensor data into session-level metrics that are calculated dynamically for the active session. This generates a comprehensive 16-feature vector encompassing current (last non-null), maximum, minimum, and mean values for all four sensor types evaluated over the entire active session up to the prediction request:

- **Temperature**: `currentTemperature`, `maxTemp`, `minTemp`, `meanTemp`
- **Humidity**: `currentHumidity`, `maxHumidity`, `minHumidity`, `meanHumidity`
- **CO2**: `currentCO2`, `maxCO2`, `minCO2`, `meanCO2`
- **Light**: `currentLight`, `maxLight`, `minLight`, `meanLight`

This complete set of linearized features is passed to the `/predict` API endpoint to evaluate the user's current focus `rating`, providing the ML service with the full spectrum of environmental data.

3.6.3 Data Split and Validation Strategy

To ensure robust evaluation and prevent data leakage, we implemented a 64/16/20 split for training, validation, and testing:

1. **Initial Split**: The dataset is split into an 80% development set (training + validation) and a 20% hold-out test set.
2. **Validation Split**: The development set is further split, reserving 20% of it (16% of the total data) for validation and hyperparameter tuning, leaving 64% for model training.

Both splits employ **stratified sampling** based on the target `rating` variable to guarantee that all three subsets maintain the same proportional distribution of user ratings. However, a dynamic fallback is implemented to temporarily disable stratification if extreme class imbalances are present (e.g., if a rating class has fewer than two samples). The final models are evaluated primarily using **Accuracy** and **Macro F1-score**.

Initially ideal test set would be the one taken from the real usage of the system (both iot and frontend) but since not enough data was received this could not have been accomplished.

3.7 Frontend Implementation

/* Include a representative React component illustrating a key implementation decision — e.g. a data-fetching hook, a chart component, or an API call. */

3.7.1 Core Features Implementation

Authors: [Cristina Matei]

The frontend implements the main user-facing workflows of the system: authentication, dashboard monitoring, session rating, profile management, device connection, calendar planning, theme switching, and language switching. Each workflow is built as a React page or reusable component, while backend communication is kept in service files under `src/services`.

The application uses protected routes to separate public pages from authenticated pages. Login and registration are available before authentication, while dashboard, profile, and calendar pages require a

logged-in user. After login, the dashboard becomes the main page because it shows the current environment state and study suitability.

The following subsections describe the most important frontend features in more detail.

3.7.1.1 Dashboard Implementation

Authors: [Cristina Matei]

The dashboard is the main part of the frontend. It loads environment data through `DashboardService`, prepares the data for display, and shows the newest values in sensor cards. The cards show temperature, humidity, CO2, light level, and suitability level. They use the reusable `SensorCard` component, so the same structure can be reused for each sensor value.

The history graph is built with Recharts in the `SensorChart` component. It shows temperature, humidity, CO2, and light values over time. It also shows the predicted study quality on a separate axis, because the rating uses a different scale than the sensor values. The user can turn sensor lines on and off, which makes the chart easier to read when many lines are visible at the same time.

The recommendation card is based on the latest predicted study quality. Ratings 4 and 5 are shown as good conditions, rating 3 is shown as acceptable, and ratings below 3 are shown as poor. The card uses different status classes and messages for these cases, so the user does not only see a number but also gets a short explanation.

The dashboard also includes the session flow used for rating study sessions. The frontend does not create the real IoT session itself. Instead, when the user presses Start Session, the dashboard calls the backend through `SessionService` to check whether the connected device has an active session. If an active session is found, the frontend stores the session ID in state, marks the dashboard session as active, and shows the live sensor cards and chart. If no active session is found, the live view remains locked and the dashboard shows a message explaining that no active device session was found.

While the frontend session is active, a timer is started. After 60 minutes, the rating popup opens automatically. The user can also press Stop Session manually, which opens the same popup. The popup is implemented in the `SessionRating` component and requires the user to choose a rating from 1 to 5 before submitting. When the rating is submitted, `RatingService` sends the device ID, session ID, rating value, and an empty comment field to the backend. This connects the user's experience to the active study session created by the IoT device.

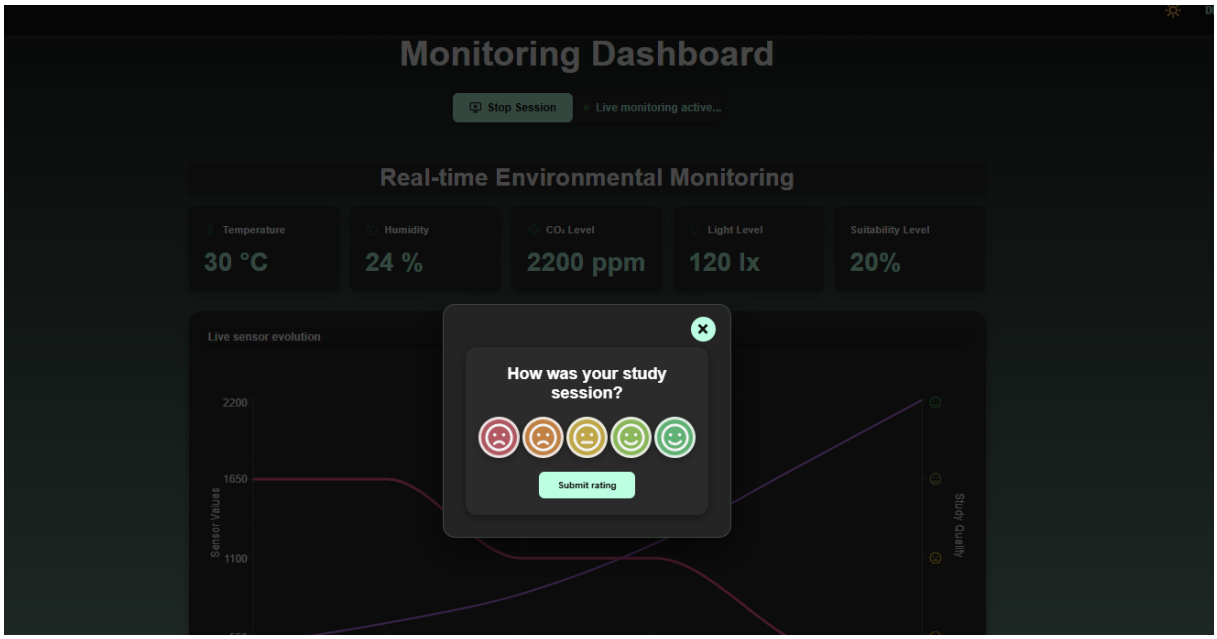


Figure 3X: Session rating popup shown when the user stops an active study session. The user selects a rating from 1 to 5, which is submitted together with the device and session identifiers.

Loading and empty states are also handled in the dashboard. While the page checks the device connection or loads data, it shows `LoadingSpinner`. If no device is connected, it shows an empty state telling the user to connect a device from the profile page. If a device is connected but no readings are available yet, another empty state is shown. This makes the page clearer when data is missing.

```
// Frontend/src/services/DashboardService.js
import { API_URL } from "./apiConfig";

export async function getDashboardData() {
  const response = await fetch(`${API_URL}/dashboard`, {
    credentials: "include",
  });

  if (!response.ok) {
    throw new Error("Failed to fetch dashboard data");
  }

  return response.json();
}
```

This service function is used by the dashboard to retrieve the latest sensor readings and predicted study quality from the Core API. The request includes credentials so the authentication cookie is sent with the request. Keeping this code in a service file separates backend communication from the dashboard UI logic.

3.7.1.2 Profile and Device Connection

Authors: [Cristina Matei]

The profile page handles both user information and device connection. When the page loads, it reads the logged-in user and then requests the profile from the backend through `ProfileService`. The user can update profile information such as university, study programme, study year, study goal, and profile picture. The page also includes password change fields with basic checks before sending the update request.

The device connection part uses `DeviceService`. The user enters a device ID, and the frontend checks if that device exists in the backend. If the device is not found, the frontend tries to register it. After this, the device ID is stored locally together with the user's email. The dashboard uses this information to decide whether it should show environment data or show the "no device connected" empty state.

This solution works for the prototype, but it is still quite simple. In a more complete version, the connection between user and device should be stored and checked fully in the backend. That would make device ownership more secure and less dependent on local browser storage.

```
// Frontend/src/services/DeviceService.js
export async function ensureDeviceExists(deviceId) {
  try {
    return await getDeviceById(deviceId);
  } catch (error) {
    if (!error.message.startsWith("Device not found")) {
      throw error;
    }

    return registerDevice(deviceId);
  }
}
```

This function is used when a user connects a device from the profile page. First, the frontend checks if the device already exists in the backend. If it exists, the device can be connected. If the backend returns

that the device was not found, the frontend tries to register it. Other backend errors are not hidden, because they may mean the API is unavailable or the request failed for another reason.

3.7.1.3 Calendar

Authors: [Marta Zrno]

The implementation of the calendar was done using the FullCalendar library, which provides a fully interactive and customizable interface. The component was implemented in CalendarPage.jsx, where the it was configured with multiple plug-ins to support monthly, weekly and daily views. Additionally, it supports interactive event selection and modification.

```
<FullCalendar
  plugins={[dayGridPlugin, timeGridPlugin, interactionPlugin]}
  initialView="timeGridWeek"
  selectable={true}
  select={handleSelect} // create event
  editable={true} // drag + resize
  eventDurationEditable={true}
  events={events}
  timeZone="local"
  eventClick={handleEventClick} // edit event
```

Figure x: FullCalendar in CalendarPage.jsx

The events are loaded dynamically from the database when the calender is initialized. To be able to format them for visualization for the user, the useEffect() hook was used for asynchronous retrieval.

```
useEffect(() => {
  const loadEvents = async () => {
    try {
      const data = await getCalendarEvents();

      const formattedEvents = data.map(event => ({
        id: event.id,
        title: event.title,
        start: event.start_time,
        end: event.end_time,
        allDay: event.all_day,

        extendedProps: {note: event.note}
      }));
      setEvents(formattedEvents);
    } catch (e) {
      console.error("Error loading calendar events:", e);}};
  loadEvents();
}, []);
```

Figure x: useEffect in CalendarPage.jsx

Event listeners were implemented for user interaction with the calendar. The user is able to select a time range, which prompts the handleSelect() function to open a popup window for inserting title and additional notes.

```
// user selects a time range → open popup for new event
const handleSelect = (info) => {
  const rect = info.jsEvent
    ? { left: info.jsEvent.clientX, top: info.jsEvent.clientY }
    : { left: 200, top: 200 };

  // popup position
  setPopupPos({ x: rect.left, y: rect.top });

  // reset form
  setSelectedEvent(null);
  setTitle("");
  setNote("");
  setTimeInfo(info);

  setPopupOpen(true);
};
```

Figure x: handleSelect() in CalendarPage.jsx

If the user decides to edit, the handleEventClick() function loads event data into the form.

```
// user clicks existing event → open popup for editing
const handleEventClick = (info) => {
  const rect = info.el.getBoundingClientRect();

  // popup position
  setPopupPos({ x: rect.left, y: rect.top });

  // load event data into form
  setSelectedEvent(info.event);
  setTitle(info.event.title);
  setNote(info.event.extendedProps.note || "");
  setTimeInfo(null);

  setPopupOpen(true);
};
```

Figure x: handleEventClick() in CalendarPage.jsx

CalendarService.js holds asynchronous service functions which perform event management operations. These functions are for retrieving, creating, editing and removing calendar events using API requests.

```
export async function createCalendarEvent(eventData) {
  const response = await fetch(`${API_URL}/calendar-events`, {
    method: "POST",
    credentials: "include",
    headers: {
      "Content-Type": "application/json",
    },
    body: JSON.stringify(eventData),
  });
  if (!response.ok) {throw new Error("Failed to create event");}
  return response.json();}
```

Figure x: createCalendarEvent() in CalendarService.js

Each request includes auth credentials so that only users who have been properly authenticated can access their own calendar.

```
const response = await fetch(`${API_URL}/calendar-events`, {credentials: "include",});
```

Figure x: credentials check in CalendarService.js requests

REST API endpoints were created for getting, editing and removing events. Pydantic request models were used to handle validation of events. Database operations were done using parameterized SQL queries.

```python
@router.get("/calendar-events")
async def get_calendar_events(
    current_user=Depends(get_current_user),
    db=Depends(get_db)
):
    result = await db.execute(
        """
        SELECT *
        FROM calendar_events
        WHERE user_id = %s
        ORDER BY start_time
        """,
        (current_user["id"],)
    )
    events = await result.fetchall()
    return events
```

Figure x: GET endpoint in calendar.py

```python
# request model
class CalendarEventRequest(BaseModel):
    title: str
    note: str | None = None

    start_time: datetime
    end_time: datetime

    all_day: bool = False
```

Figure x: request model in calendar.py

3.7.2 API Integration

Authors: [Marta Zrno]

[How does the frontend communicate with the backend? Describe error handling, loading states, polling vs websocket decisions, and how ML predictions are retrieved and displayed.]

The frontend communicates with backend services through REST API requests, which are implemented with Fetch API. Fetch API is a provider of a JS interface used for making HTTP requests. Polling-based communication was selected because the IoT device transmits sensor values at fixed time intervals, which makes two-way communication unnecessary.

In order to separate API communication from frontend components and pages, a new layer was created for authentication, calendar and profile management, and other system features. This resulted in the code being reusable, modular and easily maintainable.

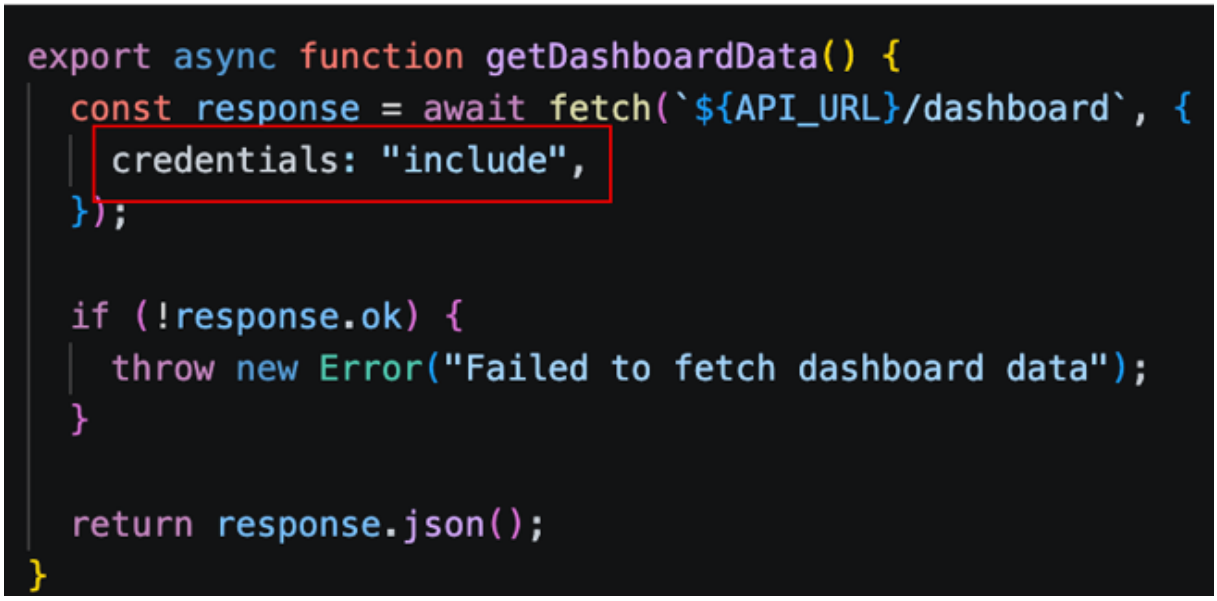
Fetch API performs the requests to the backend and is used to load the data in the webpages. Data is exchanged using JSON format, since it is human-readable and is supported by various environments.

For local and deployed environment, an API configuration was created:

```
export const API_URL = import.meta.env.VITE_API_URL || "/api";
```

Figure x: apiConfig.js

To ensure the user stays logged in during the session, their authentication credentials are included in the requests.



```
export async function getDashboardData() {
  const response = await fetch(`${API_URL}/dashboard`, {
    credentials: "include",
  });

  if (!response.ok) {
    throw new Error("Failed to fetch dashboard data");
  }

  return response.json();
}
```

Figure x: getDashboardData in DashboardService.js

Frontend components communicate with service functions through asynchronous event handlers and React hooks. These allow fast retrieval and synchronization of the data from the backend. For example, the login() function sends the user's credentials to the backend, processes the response and handles errors if the login attempt fails.

```
export async function login(data) {
  // send login request to backend
  const response = await fetch(`${API_URL}/login`, {
    method: "POST",
    credentials: "include",
    headers: {
      "Content-Type": "application/json",
    },

    // send email and password
    body: JSON.stringify({
      email: data.email,
      password: data.password,
    }),
  });
```

Figure x: login() in AuthService.js

These service functions are then used by pages asynchronously. When the backend responds successfully, the webapp states automatically updates. For example, the login page calls login(), stores the returned user data in local storage, and then redirects the user to the dashboard if the authentication was successful.

```
async function handleSubmit(e) {
  e.preventDefault();
  setError("");

  if (!form.email || !form.password) {
    setError(t.enterEmailPassword);
    return;
  }

  try {
    const result = await login(form);
    console.log(result);
    if (!result.user) {
      throw new Error("Missing login data");
    }

    localStorage.setItem("user", JSON.stringify(result.user));
    window.dispatchEvent(new Event("storage"));
    navigate("/dashboard");
  } catch (err) {
    setError(err.message || t.loginFailed);
  }
}
```

Figure x: handleSubmit() in LoginPage.jsx

In order to improve user experience, loading and empty-state components were created. These components provide visual feedback while data is being retrieved, or when no data is available.

```jsx
export default function LoadingSpinner({ text }) {
  return (
    <div className="loading-spinner-container">
      <div className="emoji-spinner">⏳</div>
      <p>{text}</p>
    </div>
  );
}
```

Figure x: LoadingSpinner() in LoadinSpinner.jsx

```jsx
export default function EmptyState({
  icon = <FolderCog size={40} strokeWidth={1.8} />,
  title,
  message,
}) {
  const displayIcon =
    typeof icon === "string" ? <FolderCog size={40} strokeWidth={1.8} /> : icon;

  return (
    <div className="empty-state">
      <div className="empty-state-icon">{displayIcon}</div>
      <h2>{title}</h2>
      {message && <p>{message}</p>}
    </div>
  );
}
```

Figure x: EmptyState() in EmptyState.jsx

MAL predictions are retrieved through malService.js. The device's sensor values are sent as JSON payloads to the /predict endpoint. This then returns a study quality prediction.

```
export async function getPrediction(data) {
  const response = await fetch(`${MAL_API_URL}/predict`, {
    method: "POST",
    headers: {
      "Content-Type": "application/json",
    },
    body: JSON.stringify(data),
  });

  if (!response.ok) {
    throw new Error("Prediction request failed");
  }

  return response.json();
}
```

Figure x: getPrediction() in malService.js

Sensor data and predictions are visualized using an interactive chart with the Recharts library. The chart displays temperature, humidity, CO_2 concentration, light level and predicted study quality values. The page contains checkboxes for each sensor, and a custom tooltip which activates a showing of data depending on the timestamp it is hovering over.

```
<LineChart data={data}>
```

```
<Line
yAxisId="prediction"
type="monotone"
dataKey="predicted_study_quality"
name="State"
stroke="#FF5DA2"
strokeWidth={4}
dot={false}
/>
```

Figure x: visualization of prediction

3.7.3 Hosting and Deployment

Authors: [Marta Zrno]

[Describe how the React app is built and hosted. Where is it deployed? How is the deployment triggered (manual push, CI/CD pipeline)? Provide the live URL if applicable.]

The application's frontend is hosted as part of the StudyHelper cloud infrastructure, as well as deployed using Docker containers. It is build using Vite- a fast frontend building tool which optimizes React's code.

The Docker build process was divided into 3 stages to separate the build from the runtime environment. In the first stage, Node.js Alpine container installed all dependencies and the product was generated using "npm run build" command. In the second stage, the generated files were copied into an Nginx container.

Figure x: Dockerfile

Figure x: Dockerfile

In the third stage, the frontend container with backend API, MAL API and database containers using Coolify. To make sure that the code that runs locally can run the same way in another environment, environment variables were configured.

Figure x: .env

Deployment was automated through GitHub Actions workflows. When new changes were pushed to the main branch, the frontend container image is automatically rebuilt and the new frontend is deployed through Coolify.

Figure x: deployment workflow

The frontend application can be accessed at: https://frontend.sep4.eduardfekete.com/

3.8 IoT CI/CD

Authors: [Jakub Maciej Baczek, Name]

3.8.1 DevOps Considerations for Embedded Development

Integrating CI/CD into an embedded C project targeting the ATmega2560 microcontroller presents challenges that are fundamentally different from those encountered in typical software projects. The most significant of these is the absence of practical emulation: unlike web or desktop applications, firmware for an AVR microcontroller cannot be meaningfully run on a standard x86 Linux host without significant behavioural divergence. This makes end-to-end automated testing on real hardware largely impractical in a CI context, as it would require physical hardware attached to the runner or a dedicated hardware-in-the-loop setup.

A further challenge is that much of the codebase is tightly coupled to hardware-dependent peripherals — UART, Wi-Fi modules, and similar — whose behaviour cannot be exercised without the target hardware. The cross-compilation toolchain adds additional complexity: building firmware for the ATmega2560 requires the AVR-GCC toolchain and PlatformIO, neither of which are part of a standard CI environment, and both of which must be installed and cached correctly to produce a reproducible build.

Automatic deployment presents a similar problem. In a conventional software project,a CD pipeline can push a build artifact directly to its destination — a server, a container registry, a package repository. For embedded firmware, deployment means physically flashing the binary onto the microcontroller, which

cannot be done without direct access to the hardware. Fully automated deployment is therefore not feasible in this context. The practical solution adopted here is to treat the compiled firmware as the deployable artifact: the pipeline produces a flashable `.hex` file and uploadsit as a GitHub Actions artifact on every successful build, ready to be downloaded and flashed to the board manually.

These constraints were acknowledged from the outset, and the CI/CD strategy was designed accordingly. Rather than attempting to run firmware on the target or simulate peripherals fully, the CI side of the pipeline focuses on two concerns: automated unit testing of hardware-independent logic, compiled and run natively on the CI runner using GCC, and a firmware build step using PlatformIO to verify that the codebase compiles correctly for the ATmega2560 target. The CD side is reduced to producing and publishing the `.hex` artifact, deferring the final flashing step to the developer. This separation allowed meaningful automation despite the inherent limitations of embedded CI/CD.

3.8.2 Tools and Pipeline

CI Pipeline

The CI pipeline is implemented using GitHub Actions and is defined in a single workflow file. It is triggered on pull requests targeting the `main` and `dev` branches. To avoid unnecessary work when unrelated parts of the repository change, the pipeline uses `dorny/paths-filter` to check for modifications within the `IOT/` directory and skips all subsequent jobs if none are found.

The pipeline is divided into three sequential jobs:

`detect-changes` — Change Detection

Runs on every pull request and uses `dorny/paths-filter` to determine whether any files under `IOT/` were modified. The `iot-test` and `iot-build` jobs are conditional on this check, and are skipped entirely if no relevant changes are detected.

`iot-test` — Testing and Coverage

Runs on an `ubuntu-latest` runner and is responsible for compiling and executing the unit test suite natively. Hardware-dependent subsystems — such as UART drivers, SPI communication, and Wi-Fi module interaction — cannot run on a host machine and were therefore isolated behind fakes and stubs using the FFF (Fake Function Framework). These fakes are placed under `test/fakes/` and are included at compile time via the `-I./test/fakes` flag, replacing real peripheral drivers with non-

operational or configurable substitutes. This allows the logic within modules such as `wifi_http.c` and `server_api.c` to be tested in isolation without any hardware dependency.

Tests are written using the Unity unit test framework for C, with test files compiled and linked against the source under test and the Unity runner. The `make coverage` target compiles all test binaries with GCC's `--coverage` flag (gcov instrumentation), executes them, and then uses `lcov` and `genhtml` to produce a HTML coverage report. `lcov` is installed via `apt-get` as a pipeline step. Third-party and test infrastructure paths (`fakes/`, `unity/`, `test/`, `/usr/`) are excluded from the coverage data to ensure only production source is measured. The resulting HTML report is uploaded as a GitHub Actions artifact named `coverage-report` for inspection after each run.

The job also requires a `secrets.ini` file containing build flags for credentials such as Wi-Fi SSID and server host. Since these cannot be stored in the repository, the file is generated dynamically in the pipeline using placeholder values sufficient for compilation and testing.

`iot-build` — Firmware Compilation

Runs only if `iot-test` succeeds and is responsible for verifying that the firmware compiles correctly for the ATmega2560 target using PlatformIO. PlatformIO is installed via `pip`, with the `~/.platformio` directory cached using `actions/cache` keyed on the hash of `platformio.ini` to avoid redundant downloads across runs. Like `iot-test`, this job also generates a `secrets.ini` with placeholder values before building. The build targets the `megaatmega2560` PlatformIO environment, and the resulting `firmware.hex` file — ready to be flashed to the microcontroller — is uploaded as a build artifact named `firmware`. This provides a verifiable, reproducible binary for every pull request that passes testing.

3.8.3 Integration into Workflow

The CI pipeline was integrated directly into the pull request workflow on GitHub. All pull requests targeting `main` or `dev` were required to either pass the `iot-test` and `iot-build` jobs or have no IoT-related changes before merging was permitted — a failing test run or a broken firmware build would block the PR. This ensured that neither regressions in testable logic nor compilation failures could be introduced into the protected branches.

Responsibility for fixing a broken build was straightforward: the author of the pull request that caused the failure was expected to resolve it. This kept accountability clear and avoided a situation where broken builds were left for others to diagnose. In practice, outright compilation failures were rare, as code was manually verified on the Arduino before being pushed. The most common failure mode was instead test failures arising from changes to modules covered by the Unity test suite.

3.8.4 Outcomes and Evaluation

The DevOps integration was largely successful within the constraints imposed by embedded development. The two-job pipeline structure — separating native unit testing from cross-compiled firmware verification — proved to be a practical and effective approach. Compilation errors targeting the ATmega2560 were caught automatically on every PR, and the unit tests provided a degree of confidence in the correctness of the hardware-independent application logic.

The use of FFF for faking peripheral dependencies worked well in practice: modules could be tested in isolation without requiring any hardware, and the fake implementations were straightforward to write and maintain. The Unity framework similarly integrated cleanly with the native GCC compilation path and the lcov coverage toolchain.

The most significant limitation of the pipeline is the scope of what could be tested automatically. Because tests run natively on an x86 host, only code that could be cleanly decoupled from hardware-specific behaviour was testable in CI. Driver-level code — responsible for directly interfacing with UART, SPI, or the Wi-Fi module — was excluded from automated testing entirely, as no meaningful substitute for actual hardware execution exists at that level. Full integration testing, including end-to-end verification of sensor readings and server communication over a real network, remained a manual process conducted on physical hardware.

Overall, the pipeline added clear value to the development workflow by catching build and logic errors early, enforcing a baseline standard for all contributions, and producing a deployable firmware artifact on every successful PR. The inability to automate hardware-level testing is an inherent property of the embedded domain rather than a gap in the implementation, and the pipeline was scoped accordingly.

3.9 Machine Learning — Models

Authors: [Piotr Junosz, Name]

In our project, we developed two distinct kinds of models to tackle different aspects of the Study Suitability problem:

1. **Session-based Models:** These models rely on chronological session data where linearization is in place. They account for the aspect of environment changes over time.
2. **Instant Measurement Models:** These models rely solely on point-in-time environmental sensor data (temperature, humidity, noise, co2, light) to predict the user's study suitability. We rigorously evaluated multiple approaches for this instant measurement prediction pipeline.

3.9.1 Model Selection

1. For the instant measurement predictions, we evaluated both regression and classification approaches since `comfortValue` is an ordinal rating (1 to 5). The models evaluated include:

- **Linear Regression (LR):** Used as a baseline regressor to establish whether linear relationships exist between sensors and comfort.
- **Random Forest Regressor (RFR):** Chosen for its ability to capture complex, non-linear interactions between environmental variables without requiring extensive feature scaling.
- **Random Forest Classifier (RFC):** Treats the 1-5 comfort ratings as distinct classes, aiming to accurately predict the exact category.
- **Gradient Boosting Classifier (GBC):** An advanced ensemble technique evaluated for its ability to sequentially correct prediction errors, offering potentially higher accuracy for the subjective ratings.
- **Multi-Layer Perceptron (MLP):** A feedforward artificial neural network evaluated to see if deep, non-linear pattern recognition could better map raw sensors to human comfort.
- **Two-Stage Pipeline:** A custom hybrid approach where Stage 1 uses Random Forest Classifiers to map raw sensor data into intermediate human "perception" values (e.g., Temperature -> "Cold", "Perfect", "Hot"), and Stage 2 evaluates both a Random Forest Regressor and a Random Forest Classifier on those intermediate perceptions to predict the final 1-5 comfort rating.

3.9.2 Training and Hyperparameter Tuning

To guarantee rigorous Data Science methodology, all instant models were trained using a strict **80% Training / 20% Test** split. We eliminated redundant validation splits to maximize training data utility.

Instead of manual tuning, we leveraged `GridSearchCV` with 5-fold cross-validation exclusively on the 80% training set to find optimal hyperparameters. To prevent data leakage, the 20% test set was locked away during the Grid Search and only evaluated once at the very end of the experiment.

The complete parameter grids searched during the tuning phase for each model were:

- **Linear Regression:**
  - `fit_intercept`: [True, False]
- **Random Forest Regressor:**
  - `n_estimators`: [100, 300]
  - `max_depth`: [None, 10, 20]
  - `min_samples_split`: [2, 5]
  - `min_samples_leaf`: [1, 3]

- **Random Forest Classifier:**
  - `n_estimators`: [100, 300]
  - `max_depth`: [None, 10, 20]
  - `min_samples_split`: [2, 5]
  - `min_samples_leaf`: [1, 3]
- **Gradient Boosting Classifier:**
  - `n_estimators`: [50, 100, 200]
  - `learning_rate`: [0.05, 0.1]
  - `max_depth`: [3, 5]
- **Multi-Layer Perceptron:**
  - `hidden_layer_sizes`: [(8,), (16,)]
  - `alpha`: [0.0001, 0.01]
  - `learning_rate_init`: [0.001, 0.005]
- **Two-Stage Pipeline (Stage 2 Regressor):**
  - `n_estimators`: [100, 200]
  - `max_depth`: [None, 10, 20]
- **Two-Stage Pipeline (Stage 2 Classifier):**
  - `n_estimators`: [100, 200]
  - `max_depth`: [None, 10, 20]
  - `class_weight`: ['balanced', None]

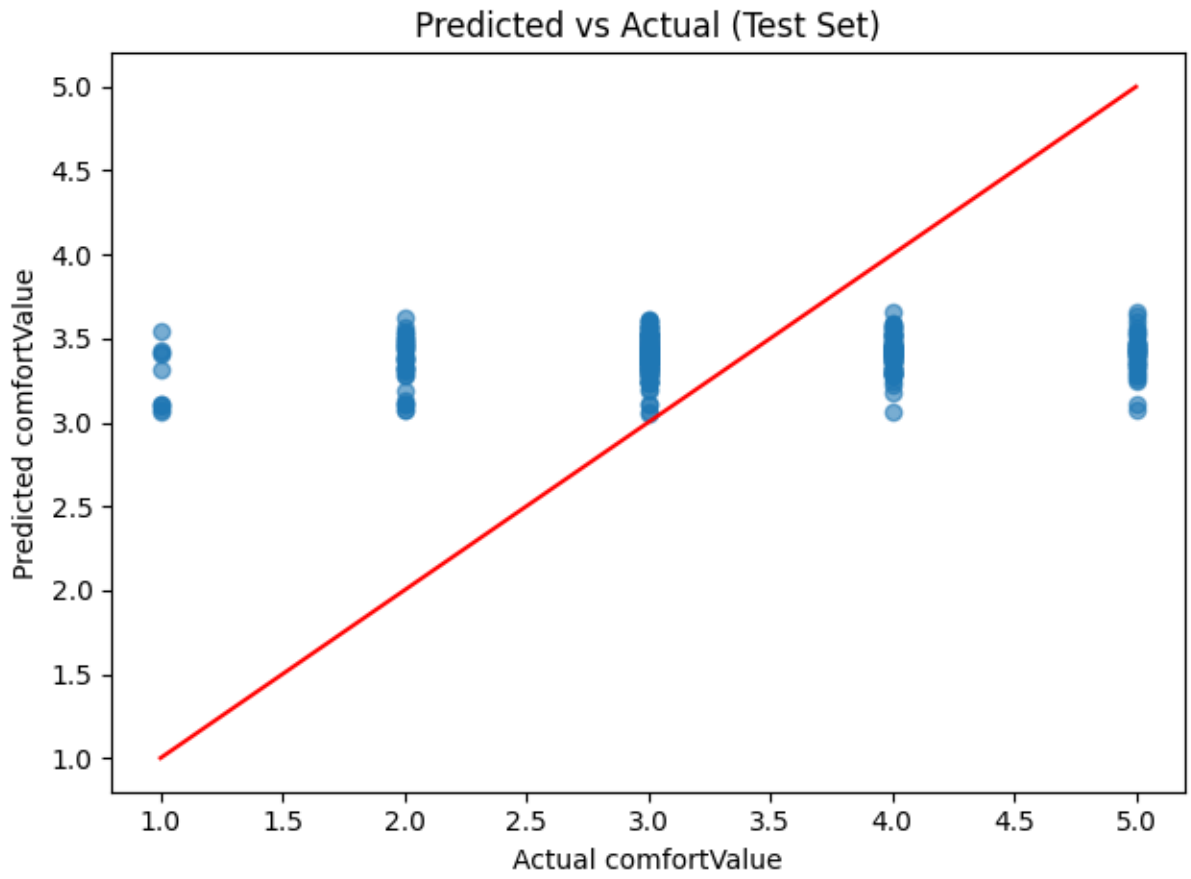
3.9.3 Model Evaluation

The models were evaluated strictly on the holdout test set to determine how well point-in-time sensor data correlates to a user's comfort. Regressors were evaluated using Mean Absolute Error (MAE), while classifiers were evaluated on Accuracy.

Model	Type	Test Metric	Result
Linear Regression	Regressor	MAE	0.749
Random Forest Regressor	Regressor	MAE	0.737
Random Forest Classifier	Classifier	Accuracy	37.2%
Gradient Boosting Classifier	Classifier	Accuracy	40.7%
Multi-Layer Perceptron	Classifier	Accuracy	46.1%

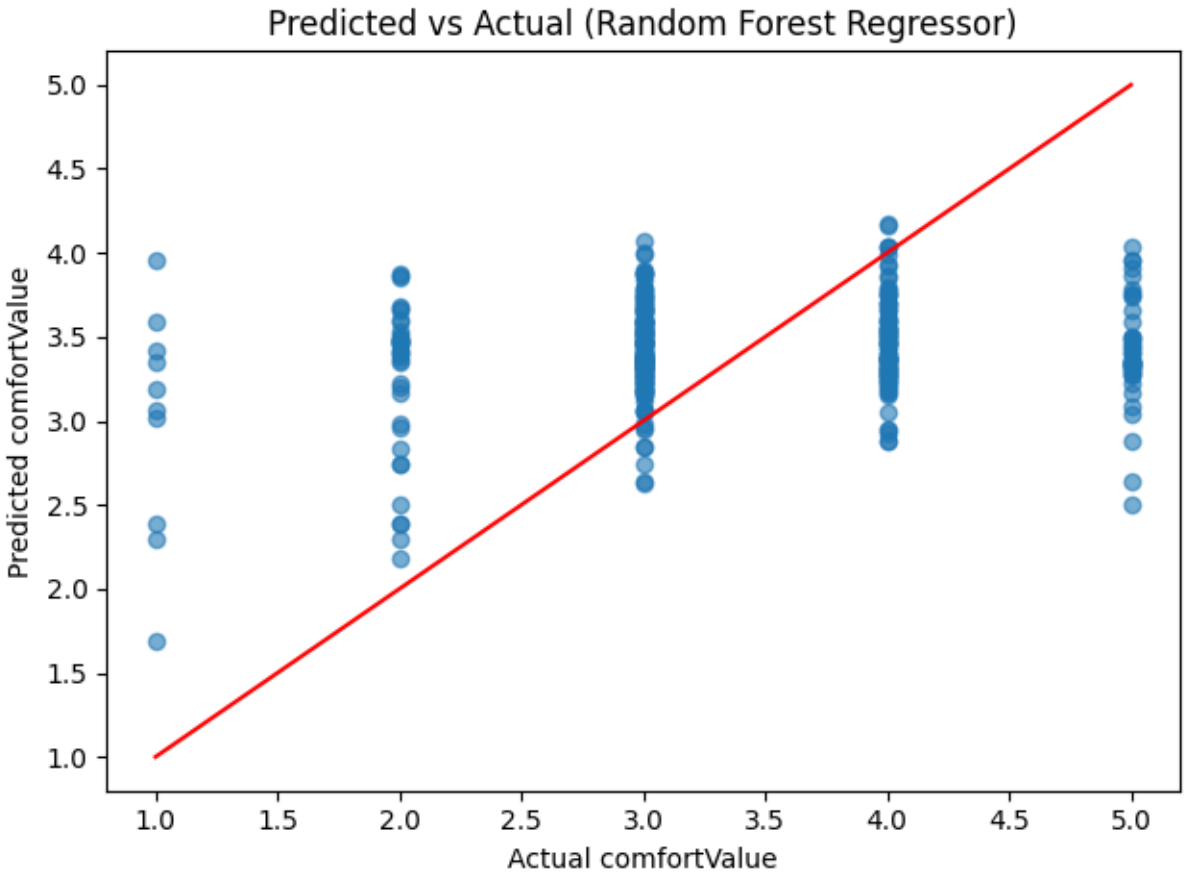
Model	Type	Test Metric	Result
Two-Stage Pipeline	Hybrid	MAE / Acc	0.667 / 48.5%

Linear Regression



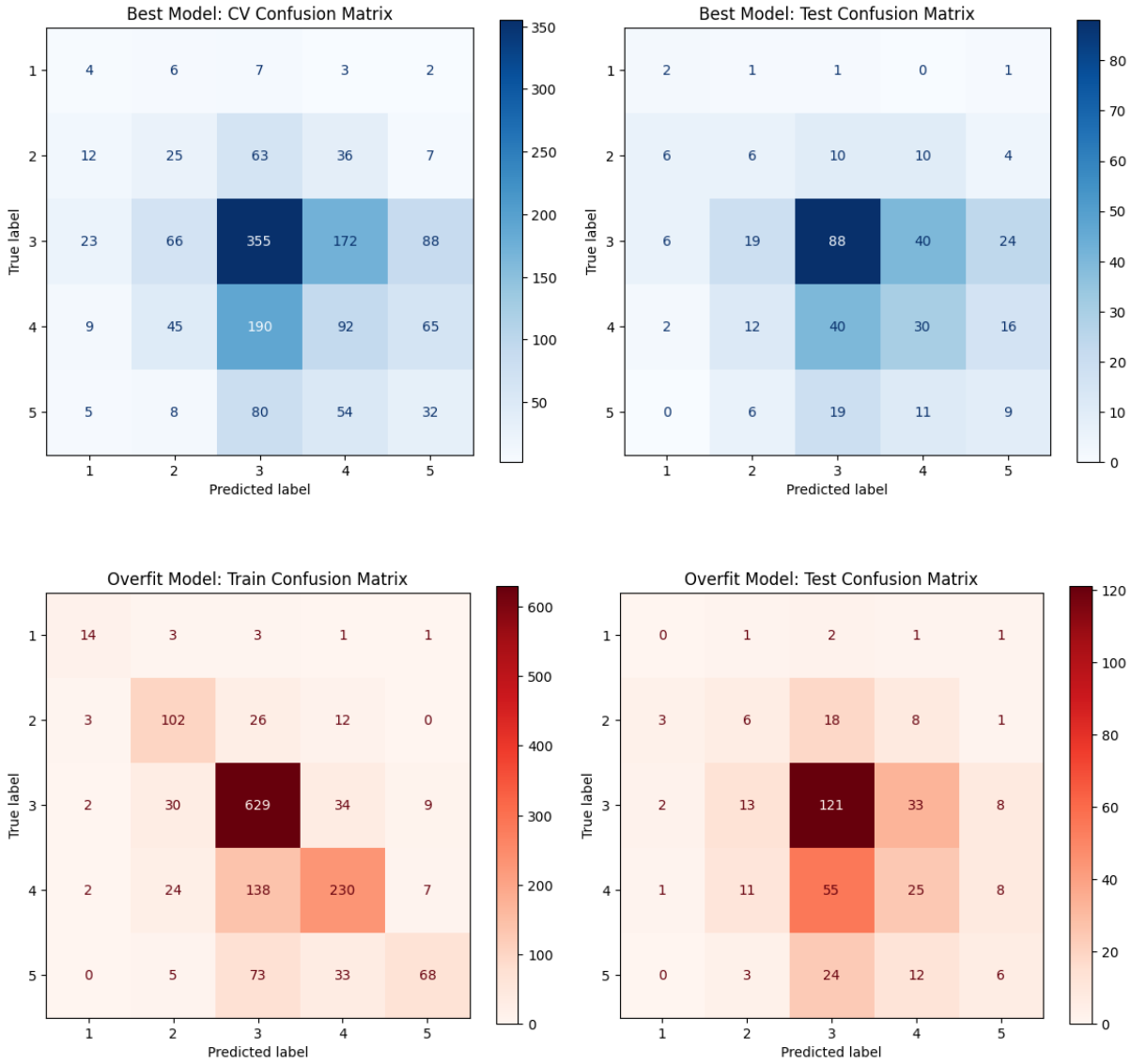
In the basic linear regression model the predicted values were all around the most common value (3) which suggested that the problem might need more complex model than linear.

Random Forest Regressor



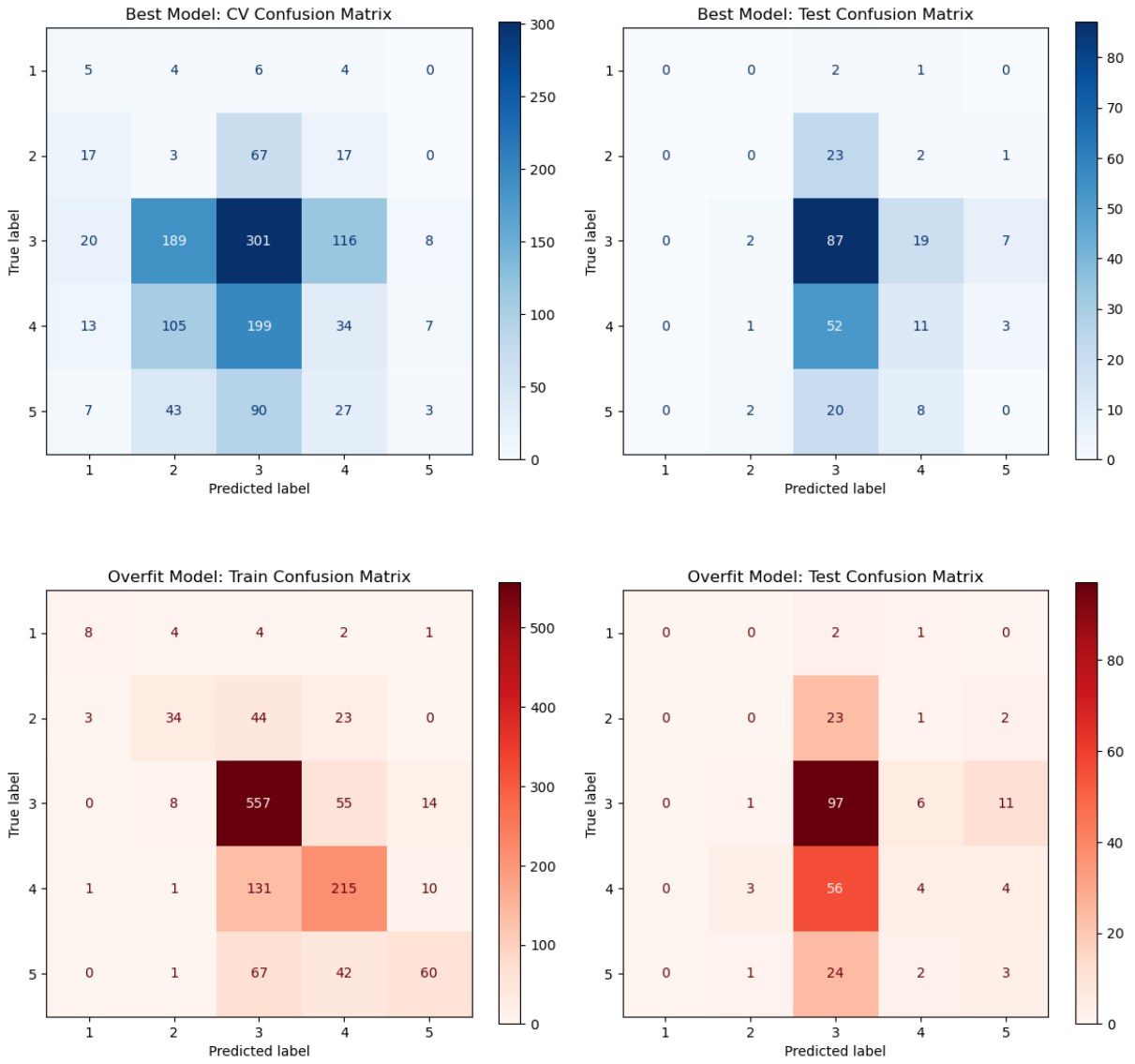
Random forest regressor did a little better - it preserved more variance and the predicted values where compressed in the middle most common value but not as much as in the linear regression, due to its ensemble nature and ability to model non-linear boundaries. However, it still could not effectively address the lack of precision in the labels - the same environmental conditions often resulted in different reported comfort levels.

Random Forest Classifier



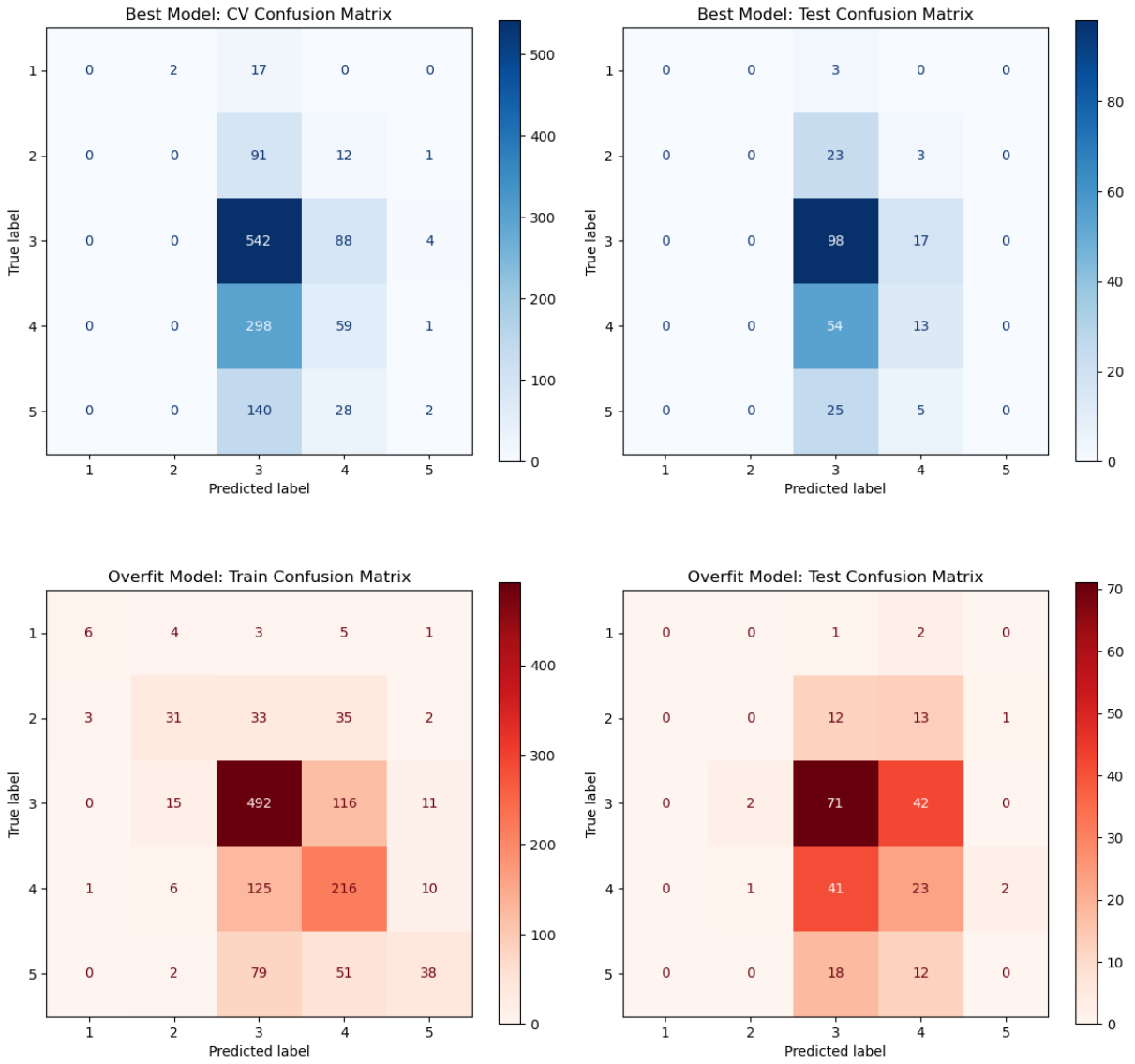
First classification model experiments were done on Random Forest Classifier and showed that classification handles this problem slightly better preserving even more variance of predicted values where also extreme classes were included sometimes. Of course the goal was to achieve this beautiful diagonal line on the test confusion matrix but unfortunatelly this was not possible, because then the model started to overfit a lot on the training set which can be seen on the oferfiting model confusion matrix above.

Gradient Boosting Classifier



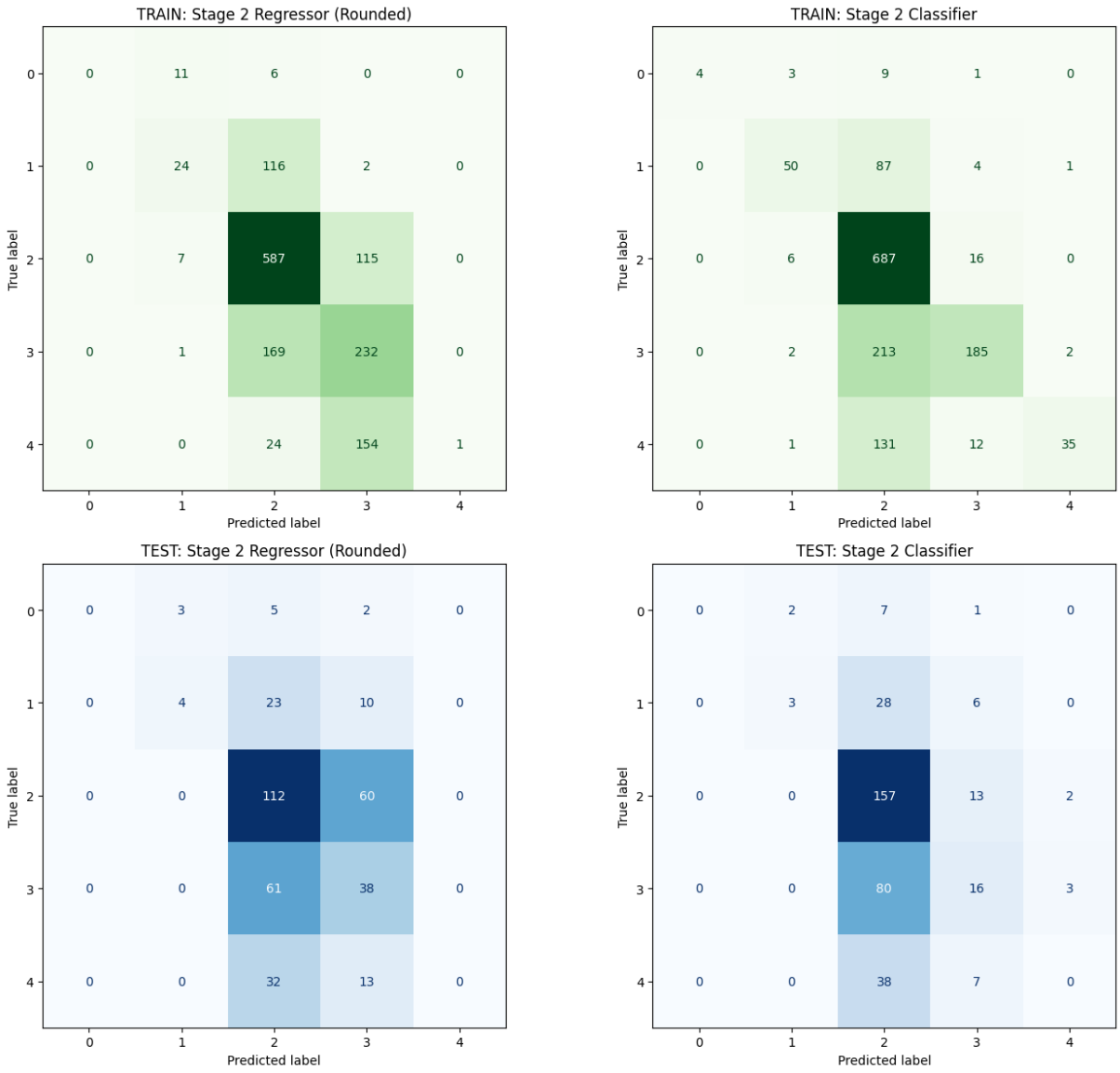
The Gradient Boosting Classifier performed slightly better than the Random Forest, reaching 40.7% accuracy. By focusing on the errors of previous iterations, it managed to sharpen the decision boundaries between the middle classes, though the extreme values (1 and 5) remained difficult to predict due to their rarity in the dataset.

Multi-Layer Perceptron



The Multi-Layer Perceptron was our best standalone classifier at 46.1%. The neural network's ability to create complex internal representations allowed it to find patterns that the tree-based models missed. However, as seen in the overfitting plots, it was very prone to memorizing the training data, requiring heavy regularization to stay useful on the test set.

Two-Stage Pipeline



The goal of this 2 stage pipeline was to use “perception” values already existing in original data that was found together with the comfort raiting. The idea was to see if this way models can predict more accurately - it turned out that not really. It lead again to overfitting on the train set and squizzing most of predictions around the mode of targets both for regressor and classifier.

Final Evaluation

As clearly visible in the confusion matrices, the instant measurement models struggled to capture a robust predictive signal. The models typically exhibited two failure modes: they either heavily overfitted to the training data, or they collapsed into predicting the majority class.

Initially, this lack of predictive power was viewed as a failure of the machine learning pipeline, leading to frustration regarding whether the feature could actually be built with data that was found and agreed between teems plan. However, upon deeper analysis, this outcome is actually one of the most valuable findings of the project. It empirically proves what we term the **"Subjectivity Paradox"**: raw physical sensor parameters (temperature, humidity, noise, etc.) are inherently insufficient to objectively predict human comfort. Two different users in the exact same room with identical environmental readings can give vastly different comfort ratings.

A universal instant-comfort model cannot exist. To solve this, future iterations of the system must rely on personalized, user-specific profiling or hardcoded rules per user rather than attempting to map objective sensor data to a generalized subjective comfort scale.

3.9.4 Result export

The best performing models were serialized into `.pkl` files as artifacts. To ensure accurate predictions, the input data must be scaled using the same parameters as the training set. Therefore, the scalers are saved as artifacts alongside the models. This allows the deployed API to build the model and expose the prediction endpoints for receiving new raw sensor arrays from the physical IoT devices.

3.10 Frontend CI/CD

Authors: [Name, Name]

3.10.1 DevOps Considerations for the Frontend

[What DevOps planning was done specifically for the React codebase? How were code ownership and review responsibilities organised?]

3.10.2 Tools and Pipeline

[Describe the CI/CD pipeline for the frontend:

- Linting and formatting (ESLint, Prettier)
- Unit and component tests (Jest, React Testing Library)
- E2E tests if applicable (Cypress, Playwright)
- Automated build and deployment to hosting
- Container usage for frontend (if applicable)]

3.10.3 Integration into Workflow

[How was the pipeline used day-to-day? Branch protection rules, required checks?]

3.10.4 Outcomes and Evaluation

[What effect did DevOps have on frontend quality and development speed? What automated checks ran on every PR? What gaps remained?]

3.11 IoT Tests

Authors: [Jakub Maciej Baczek, Name]

3.11.1 Testing Strategy for Embedded C

Unit testing for the IoT application focused on three modules containing application logic: wifi_http, responsible for establishing TCP connections and constructing HTTP requests; server_api, responsible for session management and server communication; and display_status, responsible for driving the four-digit segment display with status-specific patterns. Tests were written using the Unity unit test framework for C and compiled natively on the host using GCC, allowing them to run in CI without any physical hardware.

Hardware-dependent components — TCP socket operations, Wi-Fi driver commands, buzzer output, and display driver calls — were replaced with fakes generated using the FFF (Fake Function Framework). FFF allows individual driver functions to be substituted with configurable stubs that record call counts, capture arguments, and inject return values or response payloads. This made it possible to test application logic such as session ID parsing, retry behaviour, endpoint construction, and display output in full isolation from the underlying hardware.

The remaining codebase consists of low-level peripheral drivers and the main application loop. Drivers are inherently hardware-dependent and cannot be meaningfully tested without the target device. The main loop contains no isolated logic of its own, acting purely as an orchestrator of the other modules. Neither lends itself to unit testing, and both were verified manually on the physical hardware.

3.11.2 Unit Test Results

Module	Tests	Passed	Failed	Coverage
`wifi_http`	18	18	0	93.9%
`server_api`	36	36	0	96.6%
`display_status`	11	11	0	100%

3.11.3 Integration and System-Level Tests

No automated integration testing was implemented, as the hardware constraints discussed in section 3.8.1 make this impractical without a dedicated hardware-in-the-loop setup. Integration and system-level verification was instead performed manually throughout development. This involved flashing the firmware onto the ATmega2560 and observing end-to-end behaviour: confirming that sensor readings were correctly acquired, formatted into the expected JSON payloads, transmitted to the server, and that session lifecycle events such as session start and pulse updates were handled correctly. While informal, this process provided confidence in the integrated behaviour of the system and complemented the unit-level coverage achieved through automated testing.

3.12 Frontend Tests

Authors: [Name, Name]

3.12.1 Testing Strategy

[What test types were applied? Unit tests (component rendering), integration tests (API mocking), or E2E tests (full browser automation)?]

3.12.2 Test Results

Test Suite	Tests	Passed	Failed	Coverage
Component tests				

Test Suite	Tests	Passed	Failed	Coverage
Integration tests				

3.12.3 Responsiveness Testing

[How was responsive behaviour verified at the three required breakpoints (576px, 768px, 1200px)? Include screenshots if helpful.]

3.13 Machine Learning Tests and DevOps (MLOps)

Authors: [Piotr, Name]

3.13.1 Machine Learning Testing Strategy TODO: update after the py-cov

The Machine Learning and API (MAL) component is verified through a multi-layered testing strategy that ensures both the data processing logic and the serving infrastructure are robust.

Data Pipeline Testing We implemented unit and integration tests for the data transformation logic (e.g., `test_build_unified_environment_dataset.py`). These tests verify:

- **Merging Logic**: Ensuring that disparate datasets (IoT sensor logs, study ratings, and environmental history) are correctly joined on time-series keys.
- **Preprocessing Correctness**: Validating that the MICE imputation and k-means clustering logic produce consistent outputs without introducing data leakage.
- **Schema Validation**: Ensuring the final processed dataset matches the input requirements of the Random Forest model.

API and Model Serving Testing To verify the serving layer, we use `pytest` (e.g., `test_prediction_api.py`) to test the FastAPI endpoints. These tests serve as integration checks that:

- **Endpoint Availability**: Confirm the `/predict` and health check endpoints respond correctly.
- **Model Loading**: Verify the `rf_model.pkl` artifact is correctly loaded and can produce predictions.
- **Input Validation**: Test the API's resilience to malformed or out-of-range sensor data.

- **Inference Correctness**: Validate that the prediction output matches the expected schema and logical bounds of the Study Suitability Rating.

3.13.2 MLOps Considerations

The MAL component requires a specialized DevOps approach to manage the lifecycle of both code and serialized model weights. The primary challenge is ensuring that changes to the data processing logic in `ml_pipeline/` are always compatible with the model artifact committed in `models/`. Unlike traditional software, the "build" artifact in MLOps includes both the code and the serialized model weights (`rf_model.pkl`).

3.13.3 Tools and Pipeline

The MLOps pipeline is automated via GitHub Actions (`mlops.yaml`) and executes the testing strategy described above on every pull request.

`test-and-train` [TODO: maybe change the name because does not inlcude "train"]Job

The pipeline automates several critical verification steps:

1. **Environment Setup**: Python 3.10 is configured with dependencies, using a PostgreSQL sidecar for realistic data integration checks.
2. **Automated Verification**: The pipeline runs the full `pytest` suite, including the data pipeline and API tests. This ensures that no code change breaks the existing model's ability to serve predictions.
3. **Model Artifact Integrity**: The workflow explicitly fails if the `rf_model.pkl` is missing or corrupted, preventing "empty" deployments.
4. **Containerized Continuous Delivery**: On successful validation and merge to `main`, the pipeline builds a Docker image (`mal-api`) and pushes it to the GitHub Container Registry (GHCR).

3.13.4 Outcomes and Evaluation

This integrated Testing and MLOps approach significantly reduced deployment risks. By coupling data transformation tests with live API integration checks, we ensured that the entire pipeline—from raw data to study suitability prediction—is verifiable and reproducible. The use of Docker images for deployment ensures that the exact environment used during CI is replicated in production.

4. RESULTS AND DISCUSSION

Authors: [Name, Name, Name]

4.1 Integrated System Results

[How does the complete system perform end-to-end? Demonstrate all three parts working together (reference the submitted demo video). What does actual sensor data look like flowing through to the frontend predictions?]

4.2 Evaluation Against Objectives

[Revisit each objective from Section 1.3. For each, state whether it was met, partially met, or not met, and support the assessment with evidence.]

Objective	Status	Evidence
IoT — measure and transmit sensor readings every ≤60 s	✔ Met	§3.2.1 sensor hardware; §3.2.2 30 s data / 5 s pulse timers; §3.5.1 driver implementation and CO_2 fallback caching; §3.5.2 HTTP transmission
Cloud Backend — persist sensor data and expose a RESTful API	✔ Met	§3.1.1 Core API architecture; §3.1.2 Docker Compose and schema init; §2.3 FR02–FR03[TODO: uncomment section when ready]; §4.6 stack stable throughout project period
Machine Learning — train a 1–5 suitability classifier and expose via API	✔ Met	§3.3.3 multi-class classification formulation; §3.6.2 16-feature session vector; §3.9.1–§3.9.3 model selection, tuning, and evaluation; §3.13.1 `/predict` endpoint verified

Objective	Status	Evidence
Frontend — display live readings and ML rating responsively	✔ Met	§3.4.1 UI/UX design; §3.4.3 breakpoints at 576 px, 768 px, 1200 px; §3.7.1 data fetching and chart implementation; §3.12.3 responsiveness testing; §2.3 FR05 [TODO: The referenced section is still not complete]
DevOps — containerise all components and enforce CI/CD pipelines	✔ Met	§3.1.2 all services in `docker-compose.yml`; §3.8.2 `iot-test` and `iot-build` jobs; §3.13.3 MLOps pipeline and GHCR publish; §4.6 zero manual deployment effort
Security — encrypt IoT-to-backend; protect frontend API endpoints	⟳ Partial	§3.1.3 JWT + bcrypt for frontend endpoints; IoT-to-backend remains plain HTTP; §2.3 NFR01 not fully satisfied; secret management via environment variables enforced in `docker-compose.yml` [TODO: the referenced 3.1.3 section is still not done, 2.3 needs to be uncommented]

4.3 IoT Performance

Sensor reads proved reliable throughout testing, with no data loss or transmission failures observed. The DHT11 is read immediately before each transmission, so temperature and humidity values are always current. The CO_2 sensor follows a request-then-read pattern across cycles due to its internal measurement delay; if a fresh reading is unavailable, the system falls back to the last cached value and continues transmitting uninterrupted.

All buffers are statically allocated at compile time, avoiding heap fragmentation on the ATmega's limited SRAM. Each HTTP request opens a TCP connection, waits up to 3 seconds for a response with watchdog resets in the busy-wait loop, then closes — keeping memory usage predictable and preventing watchdog-triggered reboots during legitimate network waits. No latency issues, sampling drift, or memory problems were observed during operation.

4.4 ML Performance

Our instant ML models ended up being very not precise. If one of the models managed to achieve accuracy around 50% it was highly overfitting and if we wanted to prevent it than accuracy was dropping down to around 37%. But most importantly that percentage did not mean that the models are correctly learning and accurately predicting on the level of 37% - it was just because the results were often squizzed into same most common class in the dataset. Basically, it was found that environmental sensor values and human subjective raiting of the room are not enough data to use as features to make usable models. It was lacking for example a feature that would suggest what type of person is assesing the raiting, because right now two different users in the exact same room with identical environmental readings can give vastly different comfort ratings.

All in all compared to a naive baseline, the team built something that actually learns, even if the subjective nature of comfort makes it impossible to get a perfect score.

4.5 Frontend Quality

[Does the application meet the functional requirements? Does it display data correctly across all required breakpoints? Are all customer-facing features accessible via the UI?]

4.6 Cloud and DevOps Evaluation

The system has no serverless workloads; all services run as containers managed by Docker Compose. Under normal use — a single device sending sensor readings every 30 seconds — the stack performed without issues throughout the project period. The PostgreSQL database, main API, ML API, and frontend all started cleanly in the correct order, served requests as intended, and retained data across restarts. The ML API consistently returned study quality predictions, and the frontend remained available throughout testing. The system was also briefly tested with three concurrent devices and performed without issues.

Deployment to Coolify triggers automatically on every push to `main`, with a concurrency lock preventing overlapping deployments. CI pipelines run path-filtered tests for the API, IoT firmware, and ML service on every pull request, meaning only relevant workflows run and feedback stays fast. Only `dev` or `fix/*` branches may be merged into `main`, enforcing a consistent workflow throughout the project. Overall, the DevOps setup reduced manual deployment effort to zero and caught regressions early.

4.7 Critical Evaluation and Limitations

[Honestly evaluate validity and reliability of your results. What are the system's remaining weaknesses? What assumptions constrain the findings? What would need to change for this to be a production-grade system? Address limitations per component where the issues differ significantly.]

MAL Evaluation The biggest hurdle we faced in the ML part was what we call the "Subjectivity Paradox" already explained in paragraphs above. Because our training data came from different sources and different people, the model often got confused when two identical sensor readings had two different comfort ratings attached to them. This essentially caps the maximum possible accuracy for any generalized model.

Also, another thing is that our team did not get enough data from our sensors and real people raitings so the team relied on datasets found on the internet such as the KETI and HomeCoach which means that the models were not trained on "our" sensor data. While we attempted to unify the data using MICE imputation and clustering, it remained difficult to account for the 'data drift' that occurs when integrating external, pre-existing datasets with measurements captured directly from our own IoT devices. If we were to take this to a production level, we would need firstly to gather more data than just environemntal sensors data and also make the system learn the specific preferences of the user sitting in front of it, rather than trying to guess based on a general average. Without this personalization, the suitability rating remains a helpful hint rather than a definitive truth.

From a frontend perspective, the final application supports the main user workflows: login and registration, dashboard monitoring, profile management, device connection, calendar planning, and session rating. The dashboard can show live sensor values, historical readings, recommendations, and the predicted study suitability value returned from the backend. The Start Session flow was also improved so the frontend does not create the real IoT session itself, but checks whether the connected device has an active backend session before unlocking the live dashboard view.

There are still limitations in the frontend implementation. The dashboard depends on the backend and IoT device having an active session. If no active session exists, the frontend can show a clear message, but it cannot start real measurement by itself. This is correct for the system design, but it also means the full dashboard flow can only be tested properly when the IoT/backend session flow is running.

The device connection flow is also still prototype-like. The frontend can check whether a device exists in the backend and save the connected device for the user, but the user-device relationship should be stored and enforced more strongly in the backend in a production version. This would make device ownership more reliable across browsers and devices.

Another limitation is that the frontend has to handle missing or incomplete data from several services. For example, if dashboard readings, prediction values, or session data are not available, the interface must show empty states instead of breaking. The current version includes loading and empty states, but a production system would need more detailed error handling and clearer recovery options for the user.

The session rating flow works as a frontend workflow, but it depends on a valid backend session ID. The popup requires the user to choose a rating before submitting, and the rating is sent with the device and session identifiers. More end-to-end testing would be needed to fully verify the complete flow from IoT session creation to frontend rating submission and database storage.

The frontend also includes automated tests for several important flows, such as login, dashboard states, active session handling, stop-session rating popup, profile device connection, language switching, theme switching, and session rating submission. A limitation is that these are still mostly component and flow-level tests. More end-to-end tests against the full deployed system would give stronger confidence that the frontend, backend, database, and IoT session flow work together correctly.

5. CONCLUSIONS

Authors: [Name, Name, Name]

[Summarise the project in 3–5 paragraphs:

1. Restate the problem and the three-part approach taken
2. What was achieved — per component and as an integrated system
3. Did the system solve the stated problem, and to what degree?
4. What is the overall answer to the problem statement?]

6. FUTURE WORK

Authors: [Cristina Matei]

[Describe the most important directions for continuing or improving the system. Be specific and actionable.]

- **IoT**: additional sensors, power optimisation, OTA firmware updates, PCB design…
- **ML**: more training data, online/incremental learning, alternative models, explainability…
- **Frontend**: mobile app, push notifications, user authentication, dashboard customisation…
- **Cloud/DevOps**: full CD with staged environments, infrastructure-as-code (Terraform), monitoring and alerting (Grafana, Prometheus), load testing…]

Frontend Future work should mainly focus on making the prototype more reliable and easier to use. The frontend could be improved with a mobile app or progressive web app, since many students would probably check the system from a phone. Push notifications could warn the user when the environment becomes poor during a session. The dashboard could also become customisable, so users can choose which cards or charts they want to see. The connected device is currently stored in the backend user profile, which allows it to be restored after logout and login. A future improvement would be to support multiple devices per user and enforce device ownership fully in the backend. This would allow users to manage several devices and would ensure that session and rating data can only be accessed for devices assigned to the logged-in account.

The system also gives recommendations but does not control physical room equipment. The frontend can inform the user about poor conditions, but it does not directly control ventilation, heating, lighting, or other actuators. This keeps the project within scope, but it also means the system supports decision-making rather than automatic environment control.

For the frontend, future work could include better deployment testing before changes are released. The team could test the production build more often, check that the deployed frontend communicates correctly with the API, and add simple checks for login, dashboard loading, and routing. This would reduce the risk of frontend changes working locally but failing after deployment.

The tests should also be expanded beyond the current automated frontend tests. Future tests could include browser-based end-to-end tests, backend integration tests, ML regression tests, and simple deployment smoke tests. This would make it easier to change the system later without breaking the existing functionality.

Some originally discussed features were not included in the final prototype. Noise measurement was considered relevant for study environments, but the final firmware does not include a reliable working noise sensor. Because of this, the frontend only displays the implemented sensor values: temperature, humidity, CO2, and light.

REFERENCES

APPENDICES

Appendix A — Source Code

Component	Repository URL
IoT	[GitHub/GitLab URL]
Cloud	[GitHub/GitLab URL]
ML	[GitHub/GitLab URL]
Frontend	[GitHub/GitLab URL]

Appendix B — API Documentation

[Link to hosted API docs (Swagger/OpenAPI), or include the endpoint specification here.]

Appendix C — Hardware Schematics

Appendix D — Additional ML Figures